

Современные нейросетевые ТЕХНОЛОГИИ ИСТОРИЯ И ПРЕДПОСЫЛКИ

ЛЕКЦИЯ №1

Список литературы

1. Ростовцев, В. С. Искусственные нейронные сети : учебник / В. С. Ростовцев. — Санкт-Петербург : Лань, 2021. — 216 с. — ЭБС Лань. — URL: <https://e.lanbook.com/reader/book/160142>. .
2. Рутковская, Д. Нейронные сети, генетические алгоритмы и нечеткие системы / Д. Рутковская, М. Пилиньский, Л. Рутковский; пер. с польск. И. Д. Рудинского. - 2-е изд., стереотип. - Москва: Гор. линия-Телеком, 2013. - 384 с. - ЭБС ZNANIUM.com. - URL: <https://new.znanium.com/catalog/product/414545>

Список литературы

1. Хайкин С. Нейронные сети: Полный курс. Пер. с англ. Н. Н. Куссуль, А. Ю. Шелестова. 5-е изд.
2. Гудфеллоу Я., Бенджио И., Курвилль А. Г93 Глубокое обучение / пер. с англ. А. А. Слинкина. – 2-е изд., испр. – М.: ДМК Пресс, 2018.

- Введение в нейронные сети. Персептрон
- Многослойный персептрон
- Обучение нейронной сети
- Нейронные сети KERAS
- Линейные сети MLP

- CNN
- Классификация
- Рекуррентные нейронные сети
- Нейронные сети в задачах предиктивной аналитики

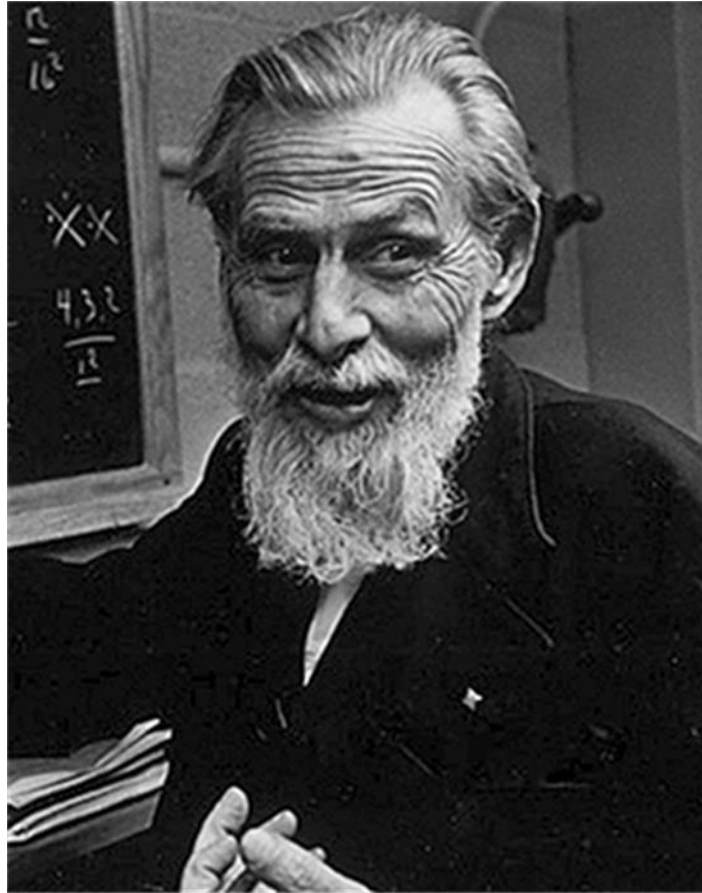
1. История открытия нейронных сетей.
2. Математическая и биологическая модель нейрона.
3. Функция активации, виды функций активации.
4. Обучение нейрона

1. История открытия нейронных сетей.
2. Математическая и биологическая модель нейрона.
3. Функция активации, виды функций активации.
4. Обучение нейрона

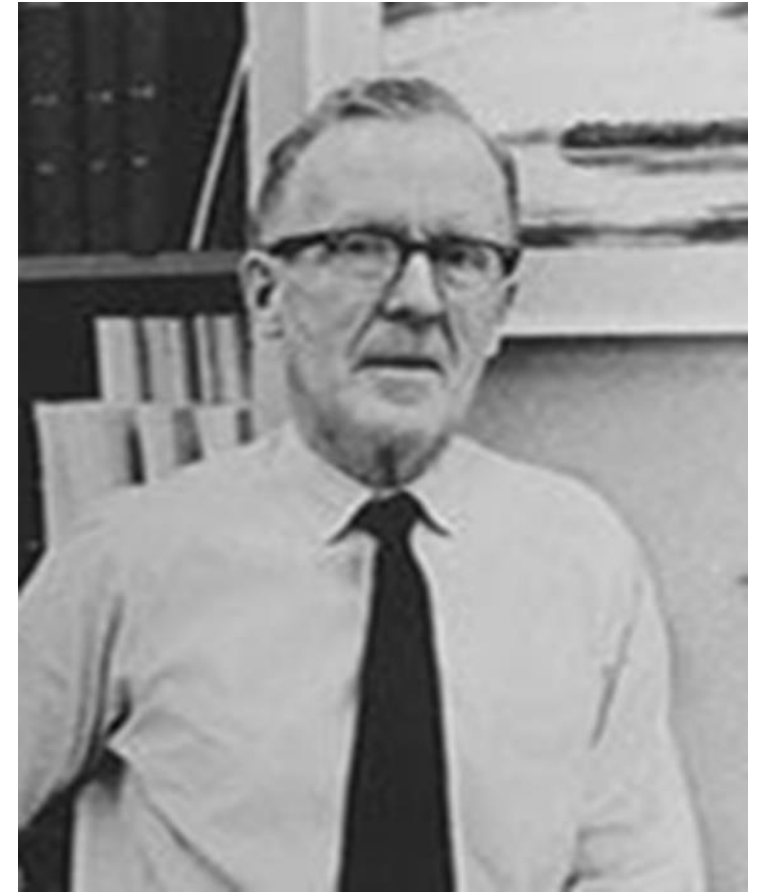
История



Уолтер Питтс,
нейролингвист

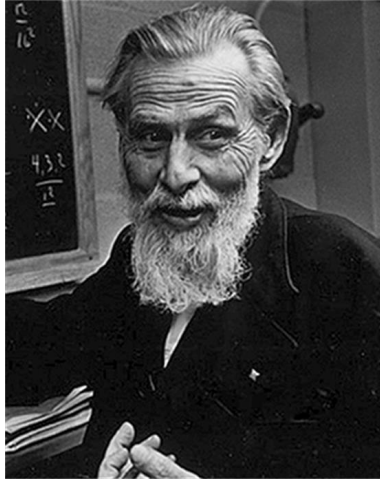
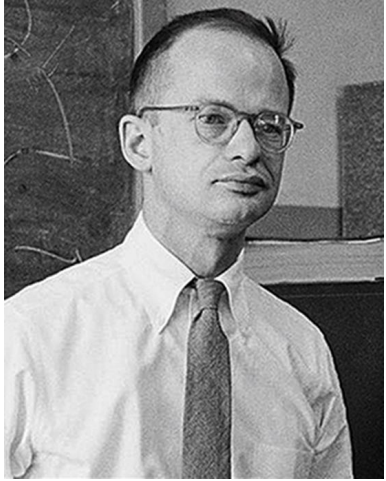


Уоррен Мак-Каллок,
нейролингвист

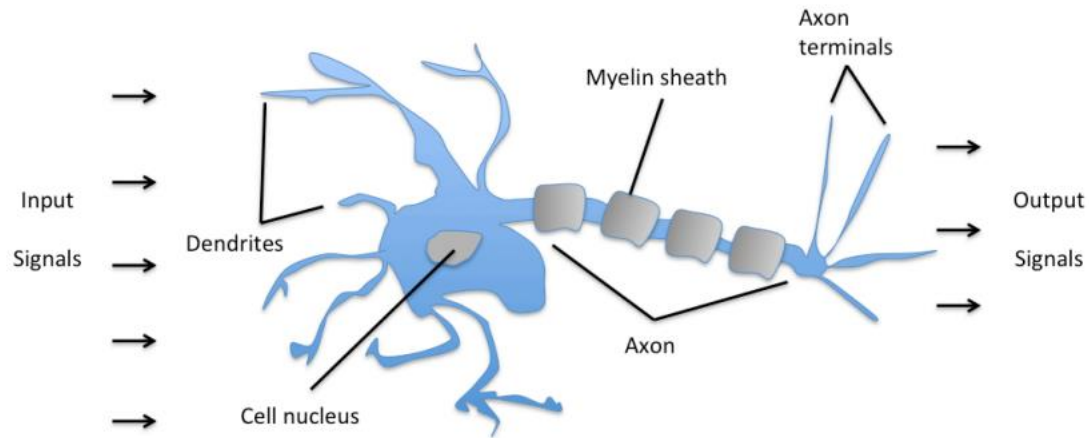


Дональд Хебб,
нейрофизиолог

История



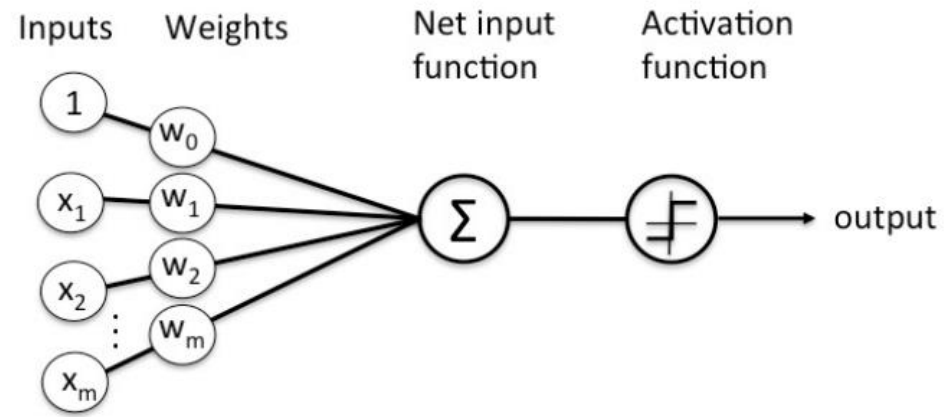
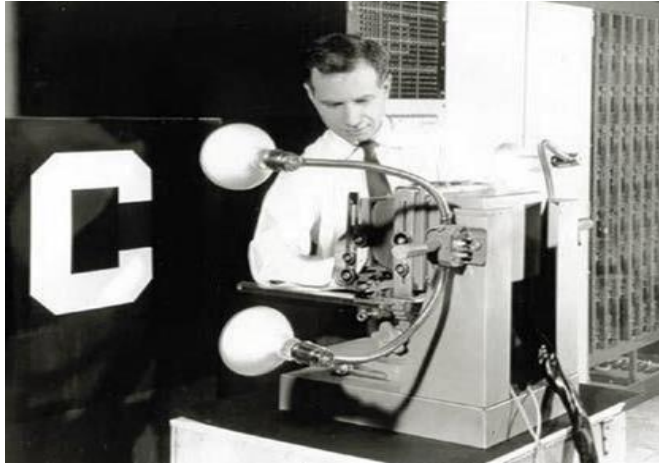
«Логическое исчисление идей, относящихся к нервной активности», 1943 г



Mathematical formulation of a biological neuron, could solve AND, OR, NOT problems

McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. The bulletin of mathematical biophysics, 5(4), 115-133

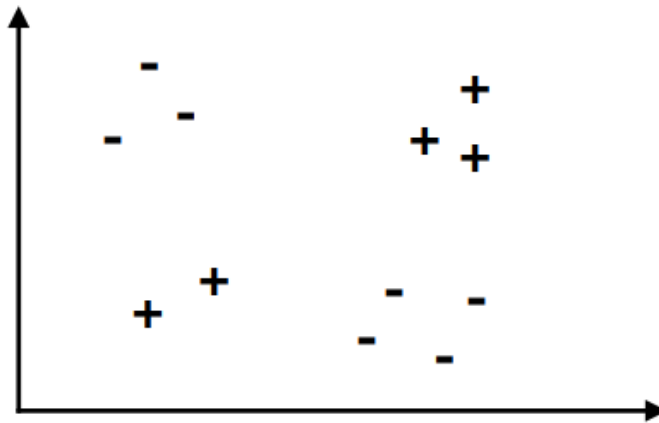
История



Rosenblatt, F. (1957). The perceptron, a perceiving and recognizing automaton Project Para. Cornell Aeronautical Laborator

История

Проблема XOR



Minsky, M. L., & Papert, S. A. (1969). Perceptrons: an introduction to computational geometry. MA: MIT Press, Cambridge

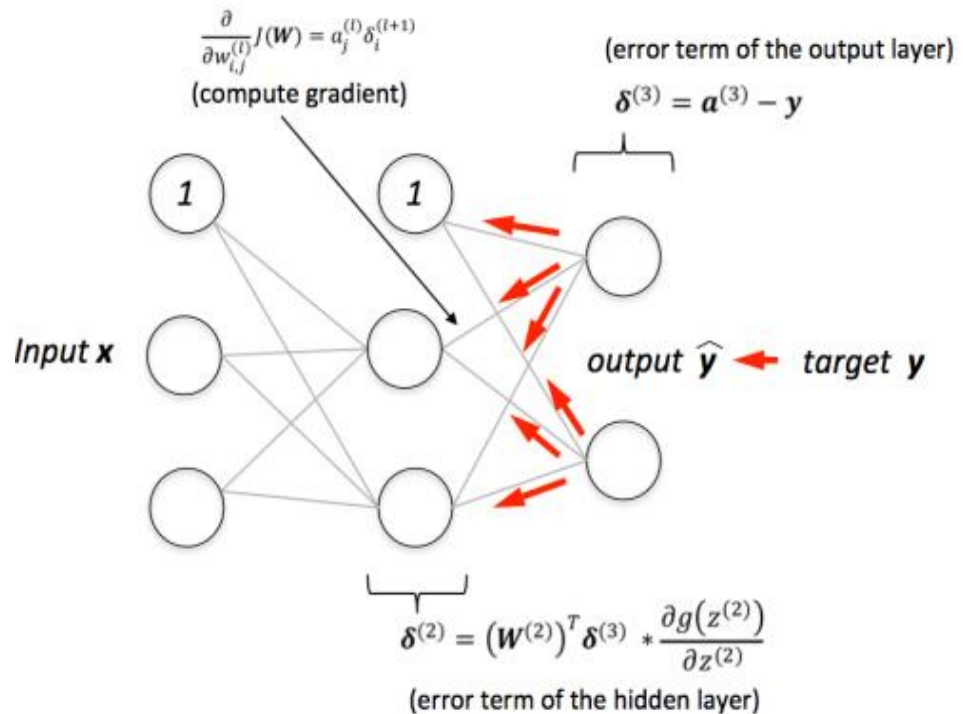
История

Backpropagation

Решение XOR проблемы:

скрытые слои, нелинейные активационные функции

- Проблема: Hard to train
- Решение: Backpropagation

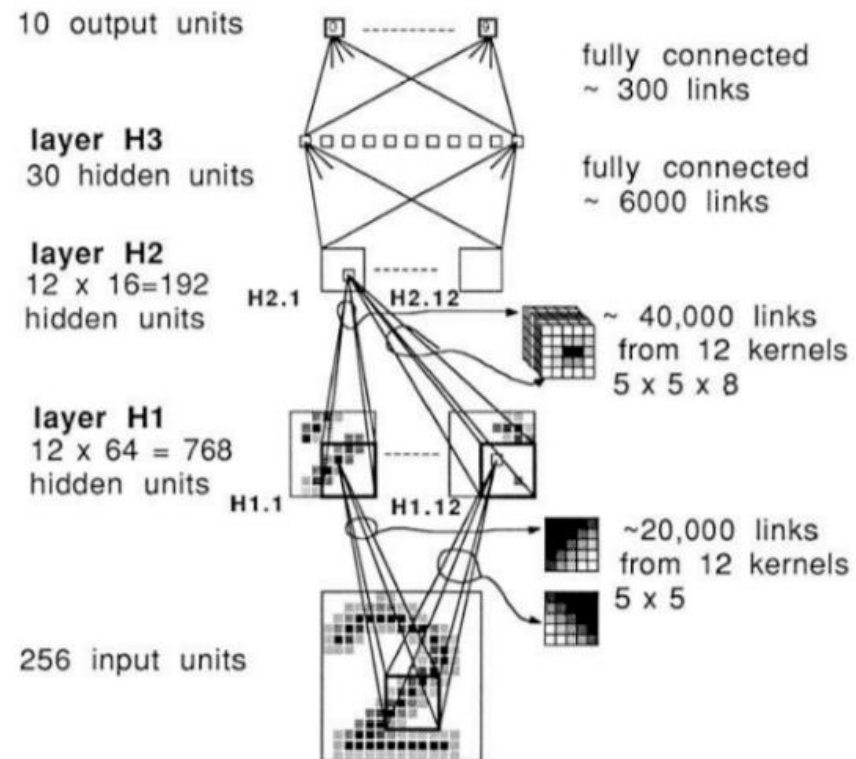


Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533



История

Convolutional Neural Networks

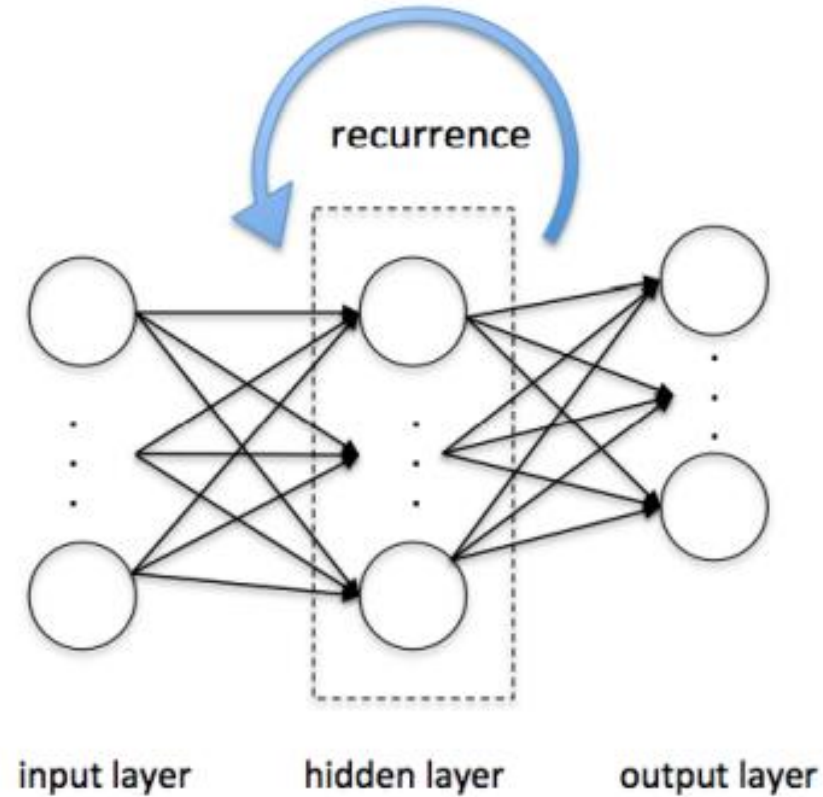


LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. *Neural computation*,

История

Recurrent Neural Networks Backpropagation

LSTM



Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8), 1735-1780

История

2nd "AI Winter" 1990's - 2000's

Решение **GPUs**

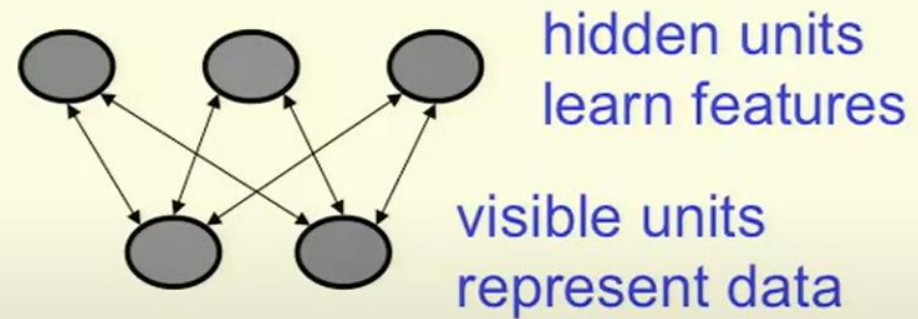


Oh, K. S., & Jung, K. (2004). GPU implementation of neural networks. Pattern Recognition, 37(6), 1311-1314

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In Advances in neural information processing systems (pp. 1097-1105).

Концепция Распределенное представление (1986).

Picture of a restricted Boltzmann Machine:



Д. Хинтон. Концепция Распределенное представление (1986).

Neural Networks Deep Learning

Rectified Linear Units

Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)* (pp. 807-814).

Dropout

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1), 1929-1958.

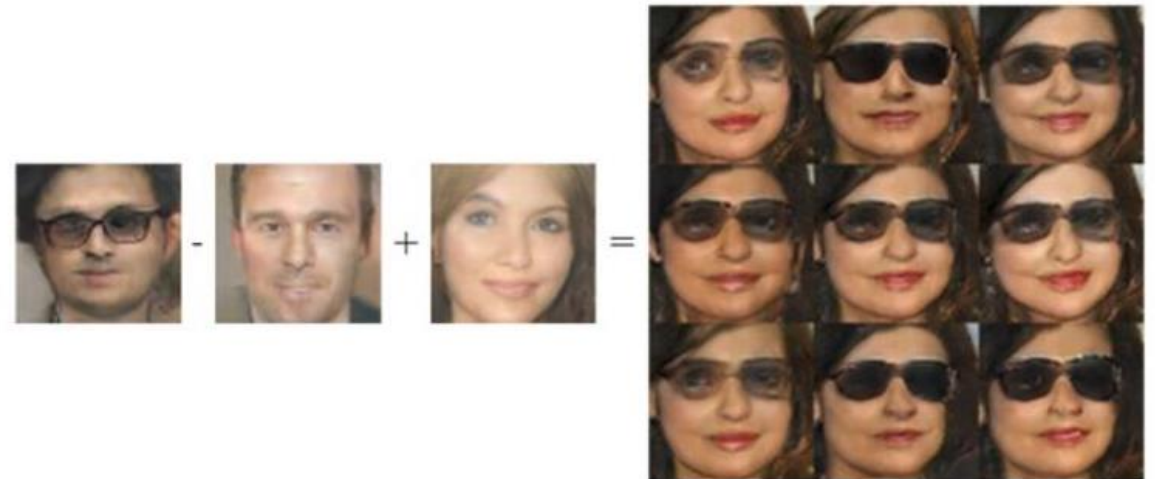
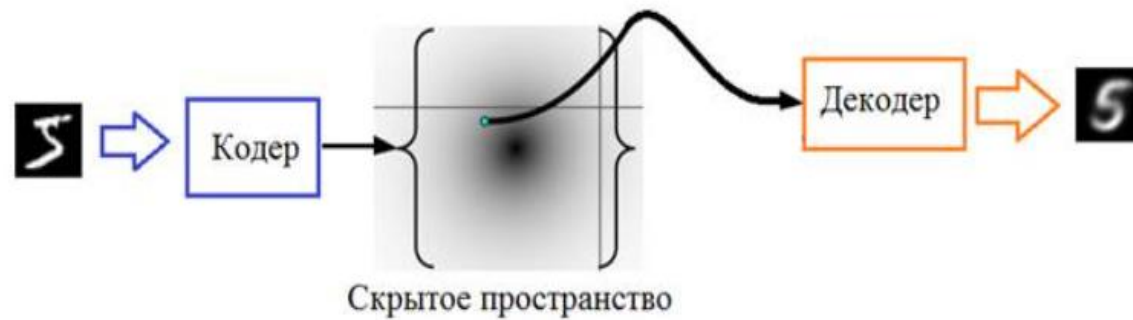
BatchNorm

Ioffe, S., & Szegedy, C. (2015, June). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In *International Conference on Machine Learning* (pp. 448-456).

GANs

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). Generative adversarial nets. In *Advances in neural information processing systems* (pp. 2672-2680).

Генеративные состязательные сети



Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). Generative adversarial nets. In Advances in neural information processing systems (pp. 2672-2680).

Geoffrey E Hinton



Yann LeCun



Yoshua Bengio

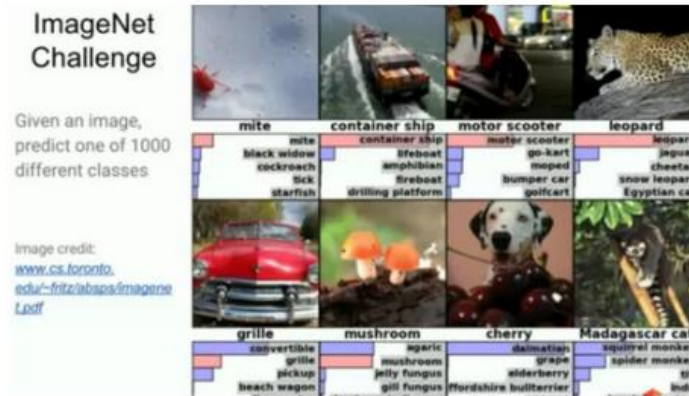
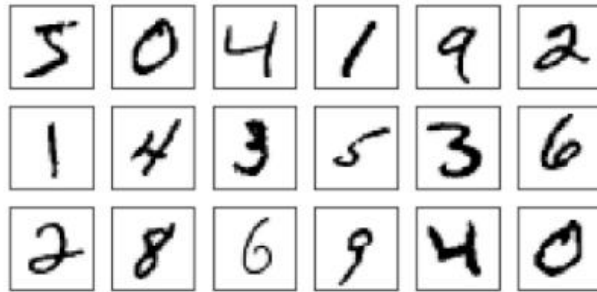


Бенжио, Хинтон и ЛеКун - победители ACM Turing Award 2018 — аналога Нобелевской премии в области вычислений — за прогресс, запустивший революцию в глубоком обучении. Их работа внесла значимый вклад в распространение технологий искусственного интеллекта.

Бенжио — профессор Монреальского университета и глава Монреальского института алгоритмов обучения;

Хинтон — профессор Университета Торонто и исследователь Google; ЛеКун — профессор Нью-Йоркского университета

Компьютерное зрение Датасеты



MNIST (1998)

- 60,000 examples, 10 classes
- features: 28x28x1
- <http://yann.lecun.com/exdb/mnist/>

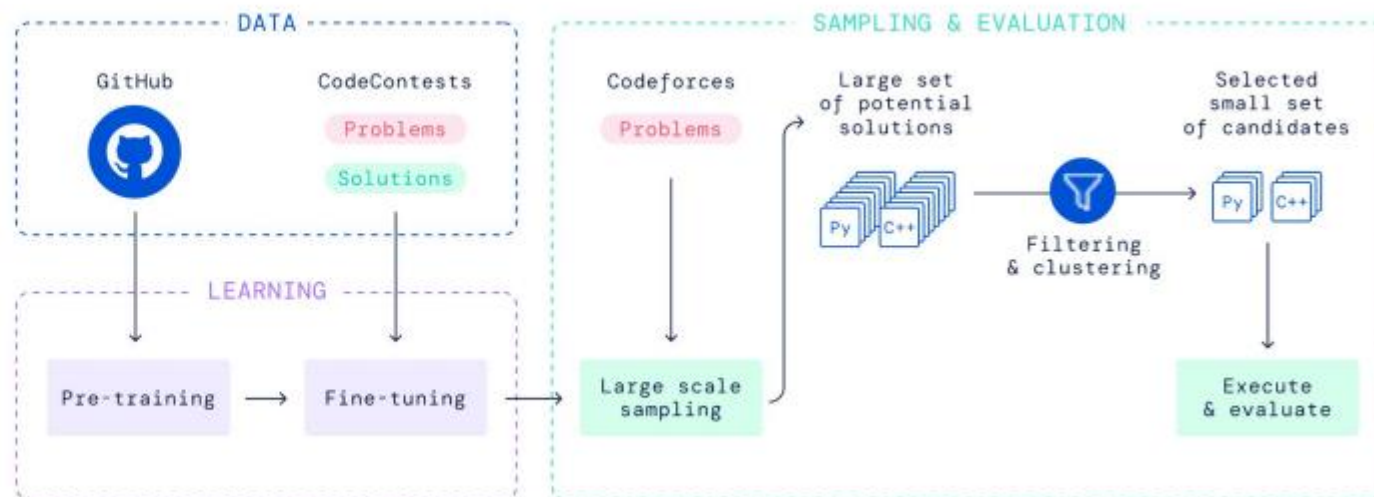
CIFAR-10/CIFAR-100 (2009)

- 60,000 examples, 10 or 100 classes
- features: 32x32x3,
- <https://www.cs.toronto.edu/~kriz/cifar.html>

ImageNet (~2010)

- ~14 million images
- features: full resolution
- <http://www.image-net.org>

Компьютерное зрение Датасеты

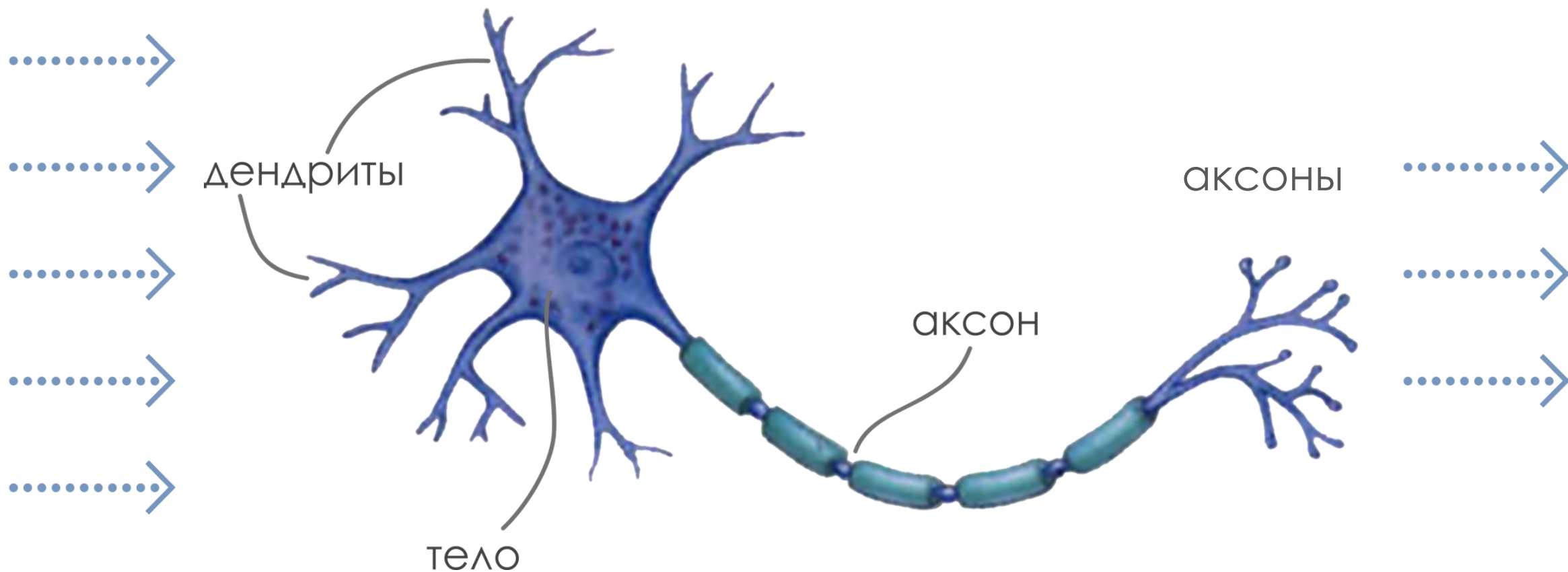


1. История открытия нейронных сетей.
- 2. Математическая и биологическая модель нейрона.**
3. Функция активации, виды функций активации.
4. Обучение нейрона

Биологическая модель нейрона

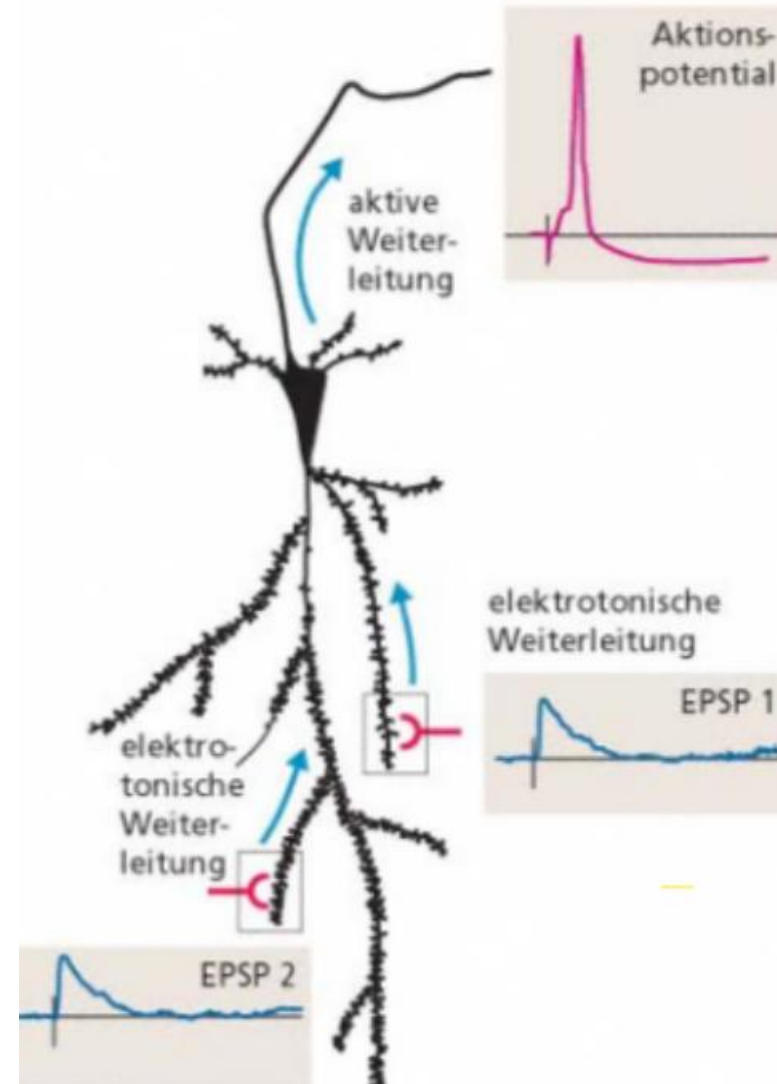
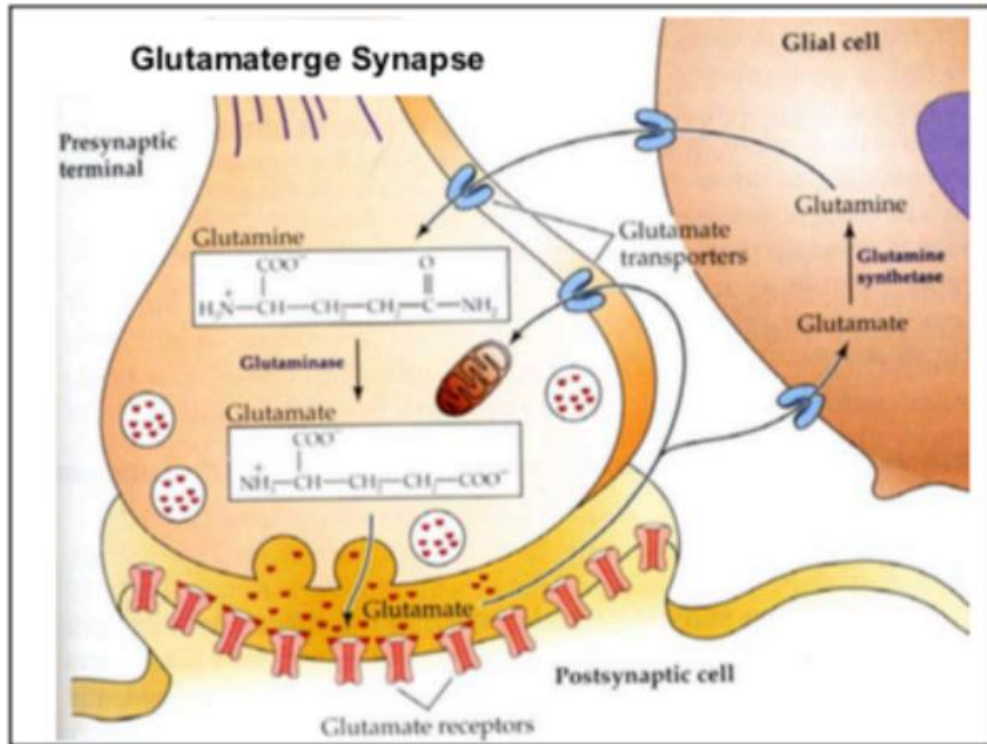
ВХОДНОЙ
СИГНАЛ

ВЫХОДНОЙ
СИГНАЛ



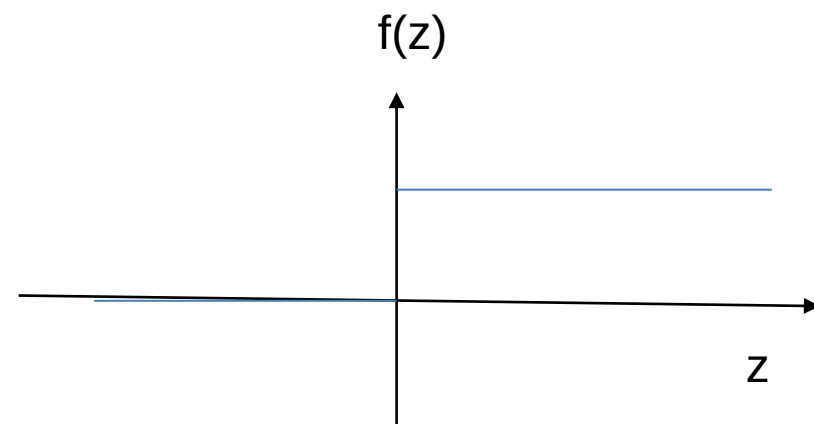
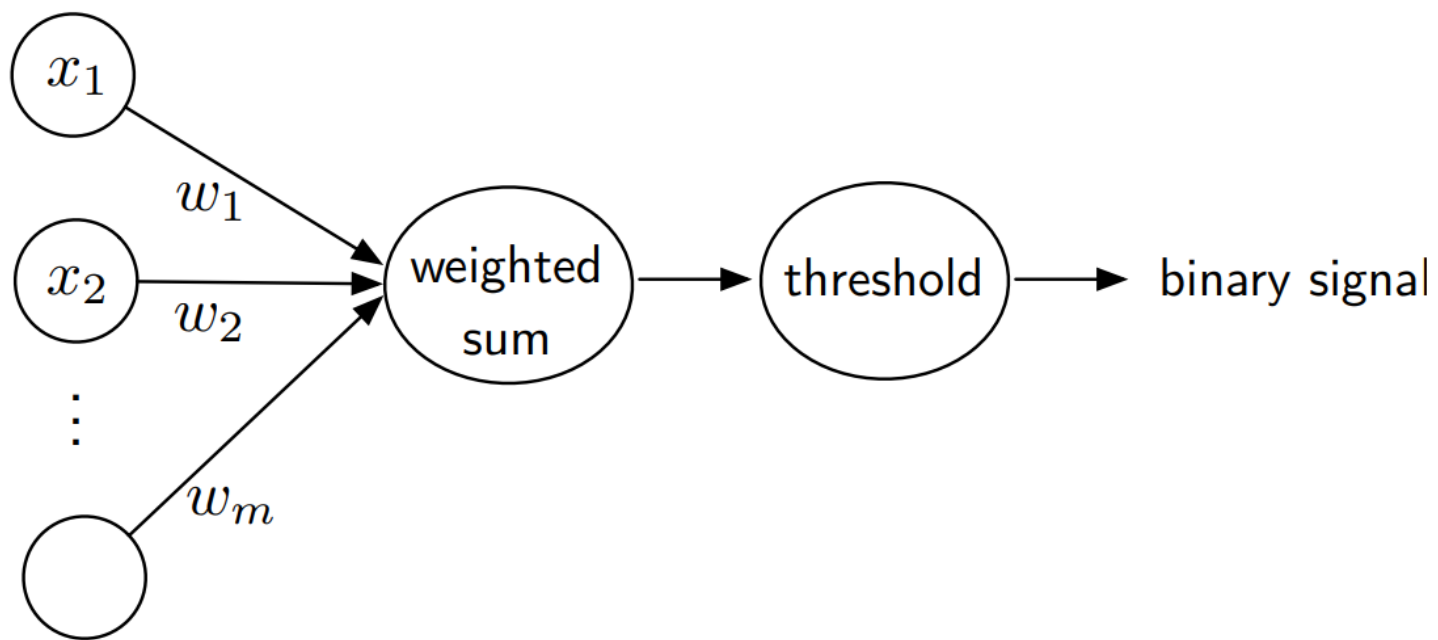
Биологическая модель нейрона

Химическая модель



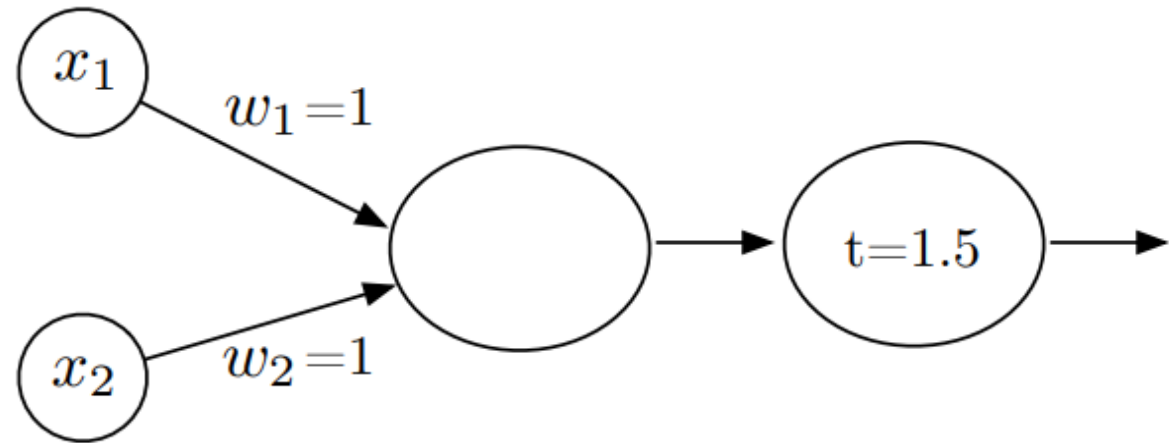
EPSP-
электричес
кий
потенциал

Нейрон МакКаллока-Пиза



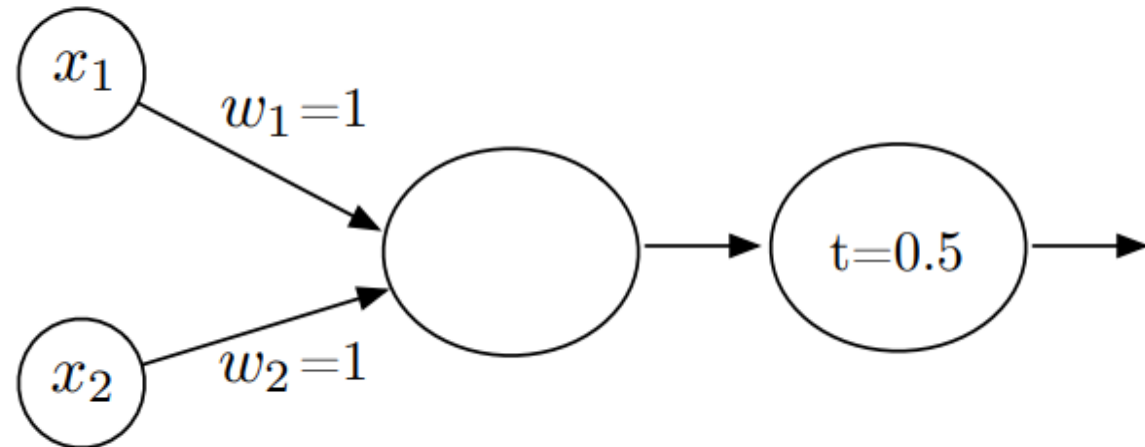
Logical AND Gate

x_1	x_2	Out
0	0	0
0	1	0
1	0	0
1	1	1



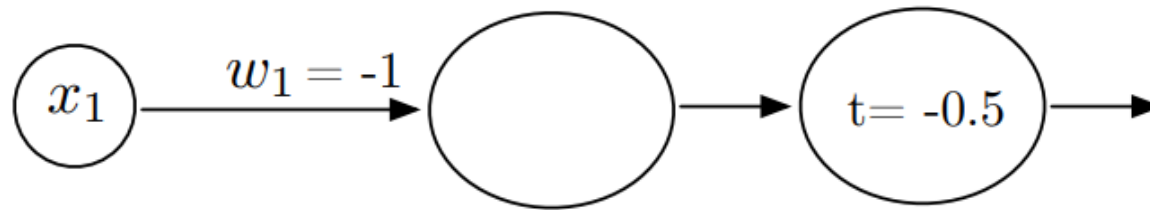
Logical OR Gate

x_1	x_2	Out
0	0	0
0	1	1
1	0	1
1	1	1



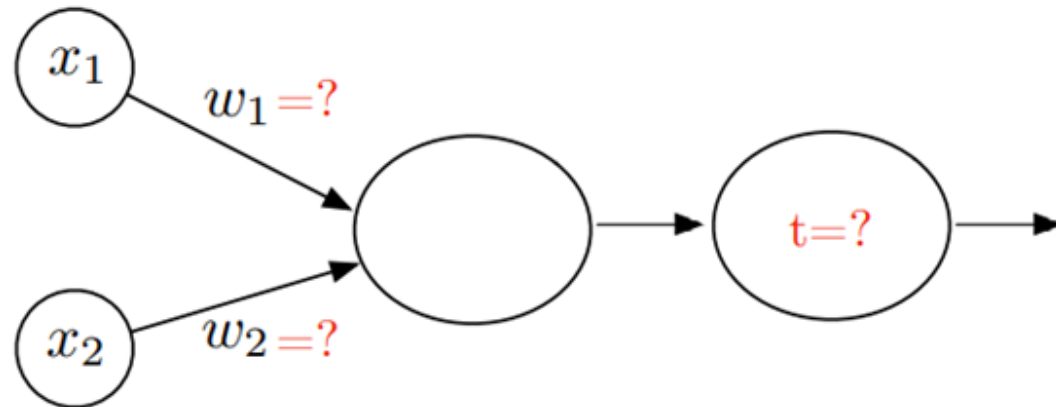
Logical Not Gate

x_1	Out
0	1
1	0

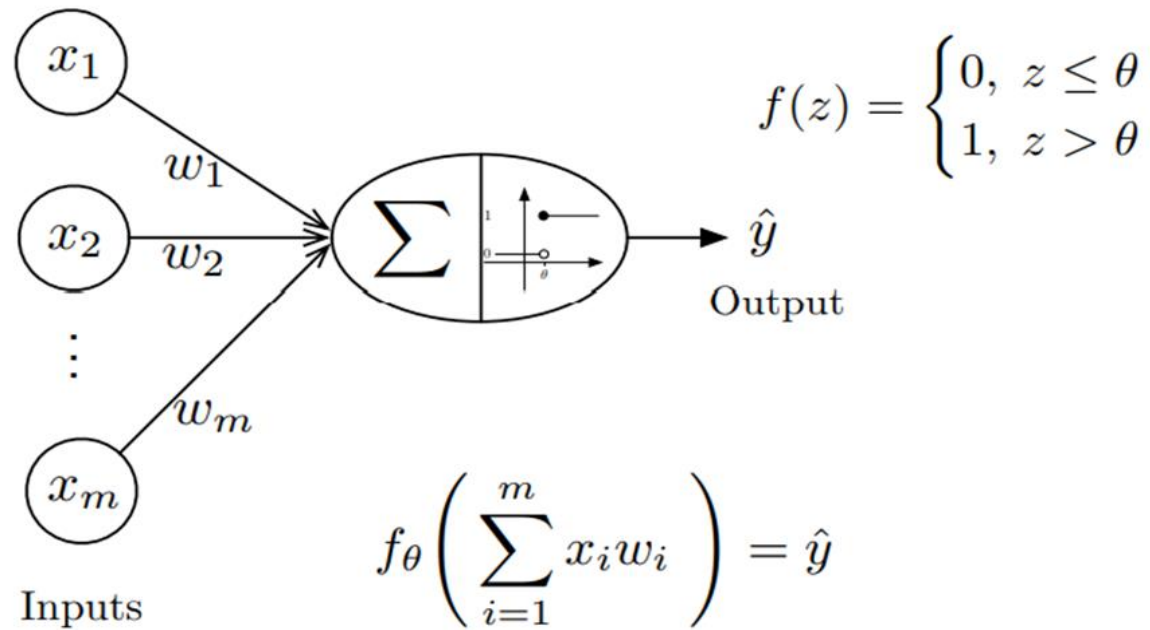


Проблема XOR Gate

x_1	x_2	Out
0	0	0
0	1	1
1	0	1
1	1	0



Математическая модель нейрона



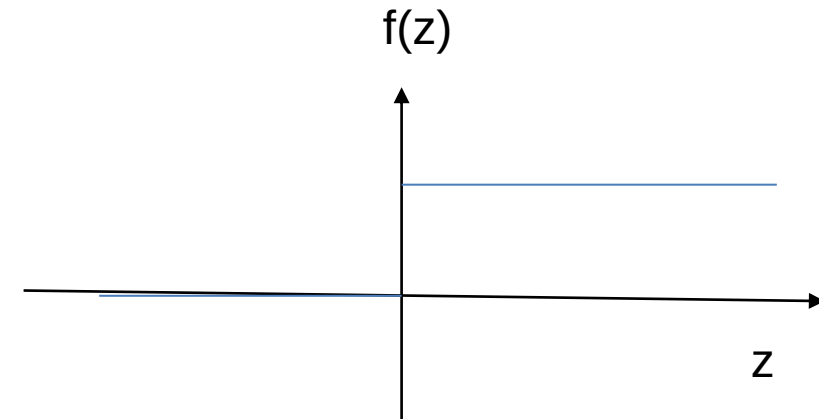
где i – номер входного нейрона

X_i – входные сигналы,

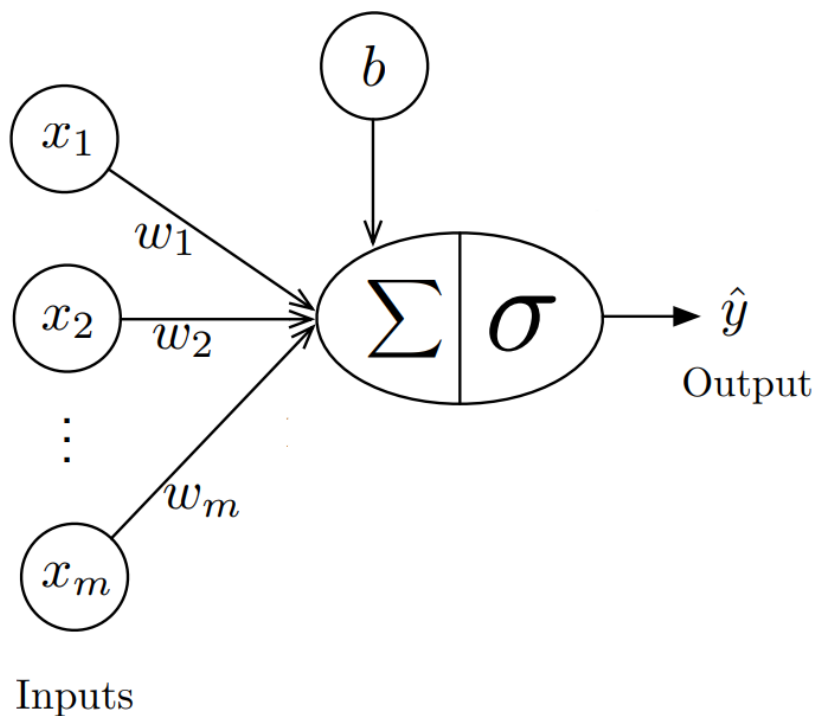
Y_j – выходной сигнал нейрона,

w_{ji} – веса синапсов,

K_j – порог нейрона, $f()$ – активационная функция.



Математическая модель нейрона

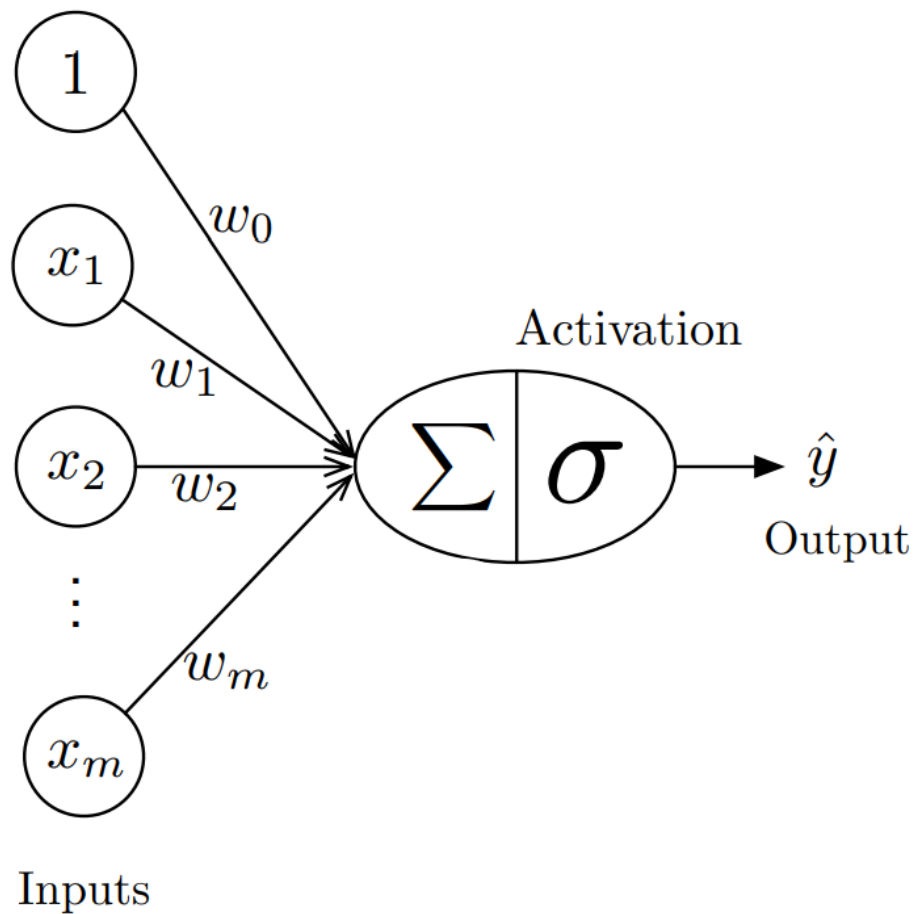


$$\sigma \left(\sum_{i=1}^m x_i w_i + b \right) = \sigma (\mathbf{x}^T \mathbf{w} + b) = \hat{y}$$

$$\sigma(z) = \begin{cases} 0, & z \leq 0 \\ 1, & z > 0 \end{cases}$$

$$b = -\theta$$

Математическая модель нейрона



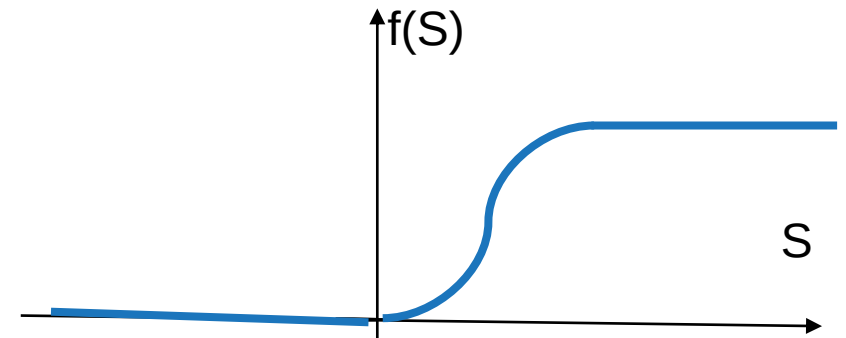
$$\sigma \left(\sum_{i=0}^m x_i w_i \right) = \sigma (\mathbf{x}^T \mathbf{w}) = \hat{y}$$

$$\sigma(z) = \begin{cases} 0, & z \leq 0 \\ 1, & z > 0 \end{cases}$$

$$w_0 = -\theta$$

Математическая модель нейрона

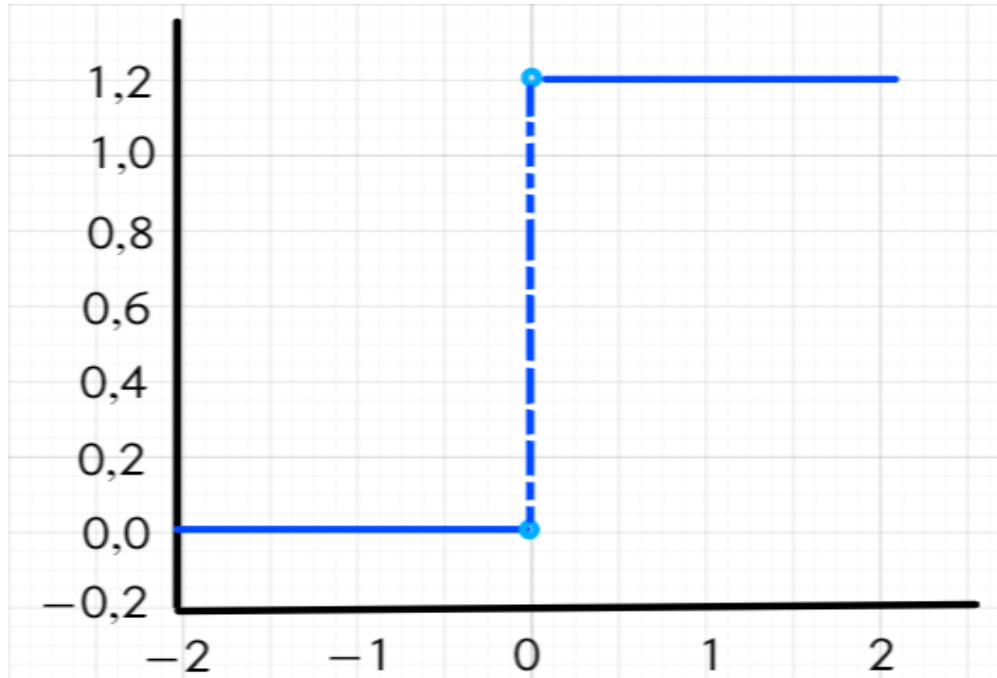
```
def sigmoid_neuron(x = None, w = None, bias=0):  
    #print('X: ',x)  
    #print('W: ',w)  
    #print('bias: ',bias)  
    s = np.dot(w, x.T )+ bias  
    #print('суммарный уровень входов нейрона на их веса: ',s)  
    y = 1 / (1 + np.exp(-s))  
    return y
```



1. История открытия нейронных сетей.
2. Математическая и биологическая модель нейрона.
3. **Функция активации, виды функций активации.**
4. Обучение нейрона

Активационные функции

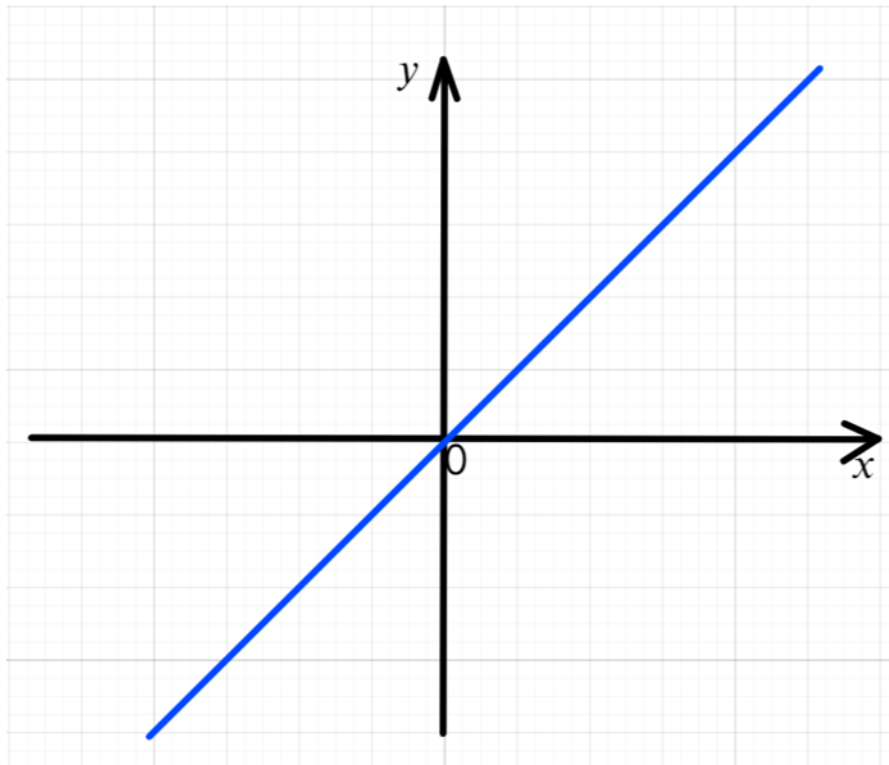
Функция Хевисайда
(binary step function)



- Бинарная классификация.
- Не работает, когда для классификации требуется большее число нейронов и количество возможных классов больше двух

Активационные функции

Линейная (Linear function)



Отличие

Область определения : $(-\infty; +\infty)$. $Y \in \mathbb{R}$,
(Классы >2)

Проблемы:

Невозможность использования метода обратного распространения ошибки. (производная — константа, при обновлении весов нельзя оценить эмпирический риск на текущем шаге).

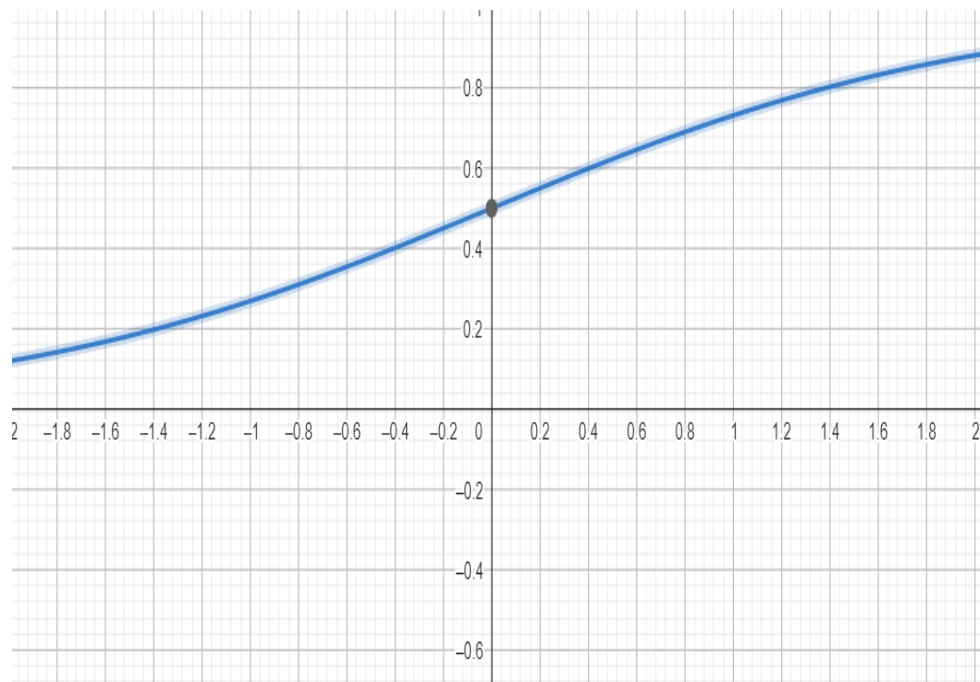
Выходное значение линейно, слои образуют линейную комбинацию, результатом которой является линейная функция. (финальная функция активации на последнем слое зависит только от входных значений на первом слое, нет смысла создавать многослойную сеть).

Задачи регрессии.

Активационные функции

Сигмоидная функция (sigmoid function)

$$y = \frac{1}{1 + e^{-x}}$$



Отличие

Преобразует выходное значение нейрона в значение из диапазона $[0, +1]$, смещает среднее значение с 0.

(LSTM-сети, логистическая регрессия)

1. Нормализация выходного значения нейрона.
2. Гладкий градиент, (предотвращает "прыжки" при подсчете выходного значения).
3. Четкое предсказание классов.

Проблема исчезающего градиента.

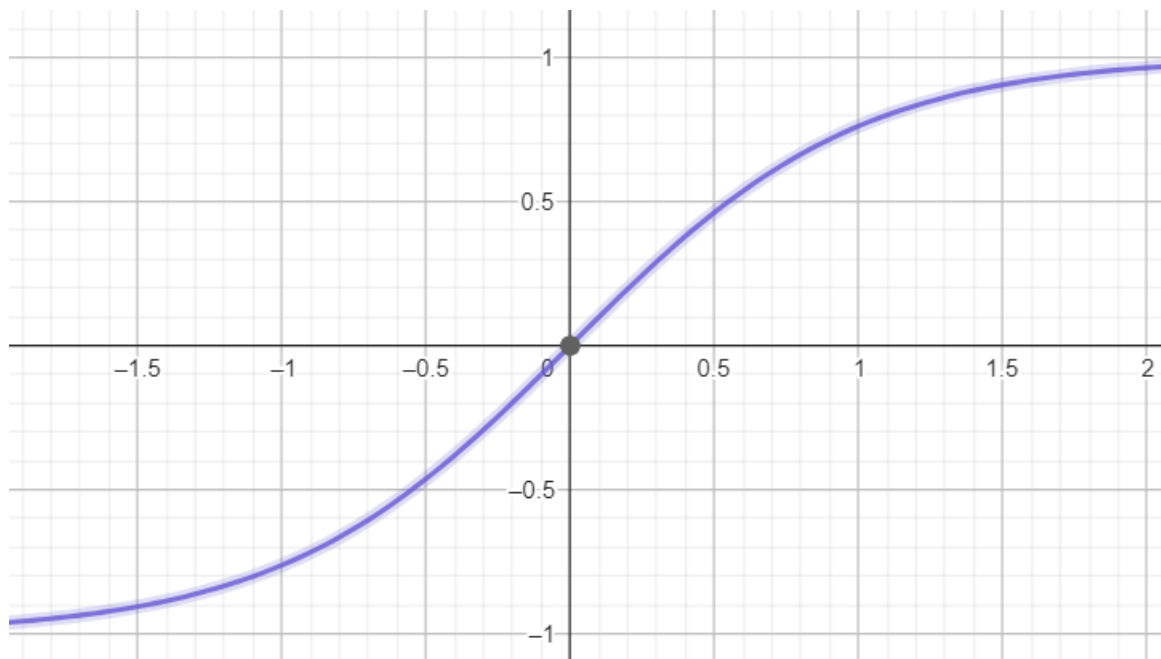
Производная крайне мала во всех точках, кроме сравнительно небольшого промежутка - сильно усложняет процесс улучшения весов с помощью градиентного спуска. усугубляется в случае, если модель содержит много слоев.

(Бинарная и многокритериальная классификация)

Активационные функции

Гиперболический тангенс (th(x))

$$y = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



Отличие

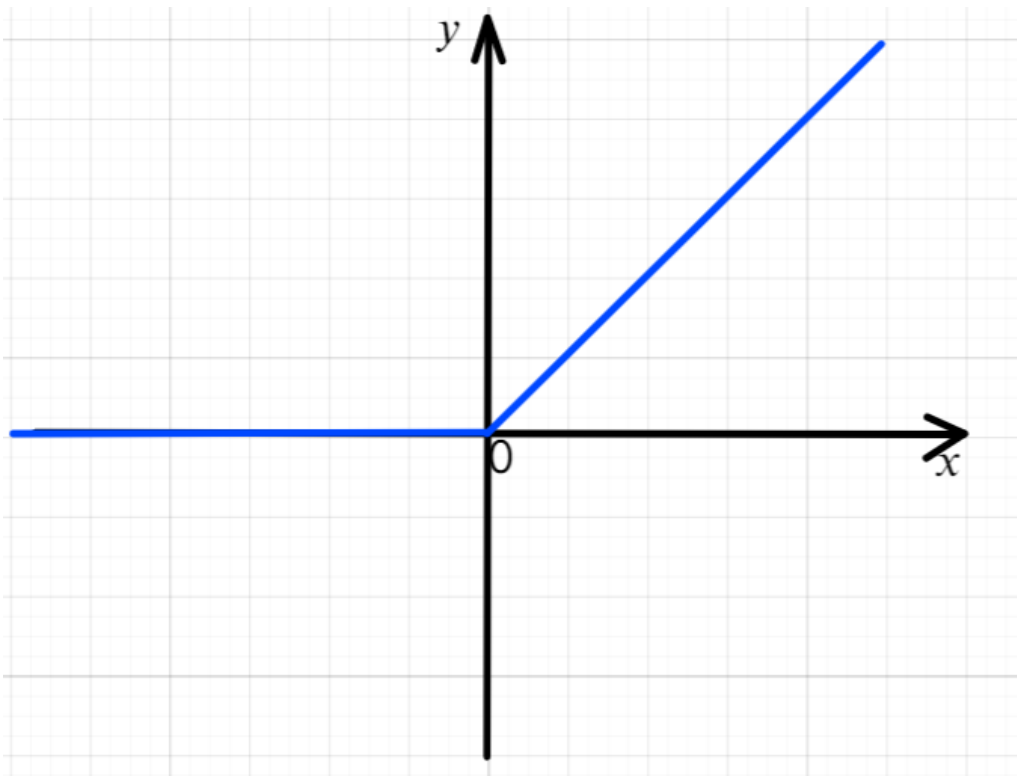
Преобразует выходное значение нейрона в значение из диапазона $[-1, +1]$, не смещает среднее значение.

1. Tanh является предпочтительнее сигмоиды в случаях, когда нет необходимости в нормализации (область определения центрирована относительно нуля, что снимает ограничение при подсчете градиента для перемещения в определенном направлении).
2. Более быстрая сходимость градиентного спуска (производная значительно выше вблизи нуля – большая амплитуда градиентного спуска).

Активационные функции

Функция ReLU (Rectified Linear Unit)

$$f(x) = \begin{cases} 0 & x < 0 \\ x & x \geq 0 \end{cases}$$



ReLU (наиболее часто используемая функция активации при глубоком обучении).

Отличие

1. $f'(x)$. Для отрицательных значений — 0, для положительных — 1.
2. Разреженность активации. В сетях с очень большим количеством нейронов с ReLU количество включаемых нейронов меньше, в силу характеристик функции - сеть станет легче.

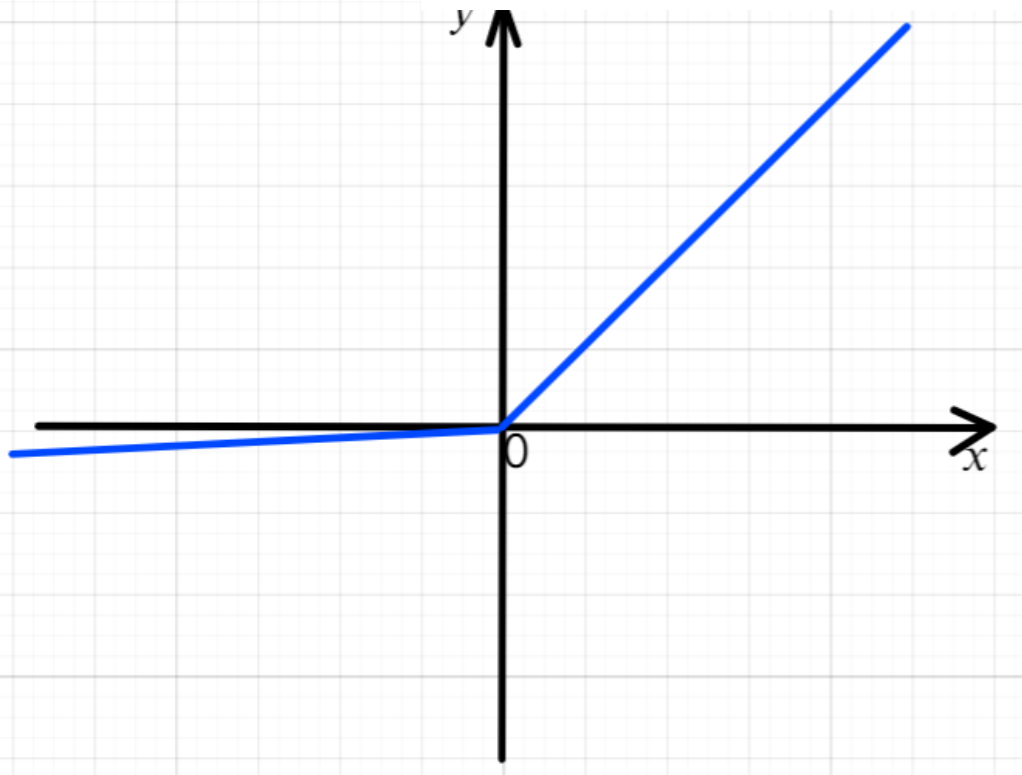
Проблема умирающего ReLU.

Так как $f'(x)=0$, градиент будет нулевым (веса не будут изменяться во время спуска, нейронная сеть перестанет обучаться).

Активационные функции

Функция Leaky ReLU (ReLU с «утечкой»)

$$f(x) = \begin{cases} 0.01x, & \text{if } x < 0 \\ x, & \text{otherwise} \end{cases}$$



Leaky ReLU помогает добиться ненулевого градиента при отрицательных значениях.

Недостатки:

1. Сложнее считать производную, по сравнению со стандартным подходом (так как значения уже не равны нулю), что замедляет работу каждой эпохи.
2. Сложная настройка гиперпараметра – углового коэффициента.
3. Проблема затухающего градиента.

При слишком большом количестве слоев градиент будет принимать очень маленькое значение, постепенно уменьшаясь до нуля.

Нестабильность в глубоких сетях.

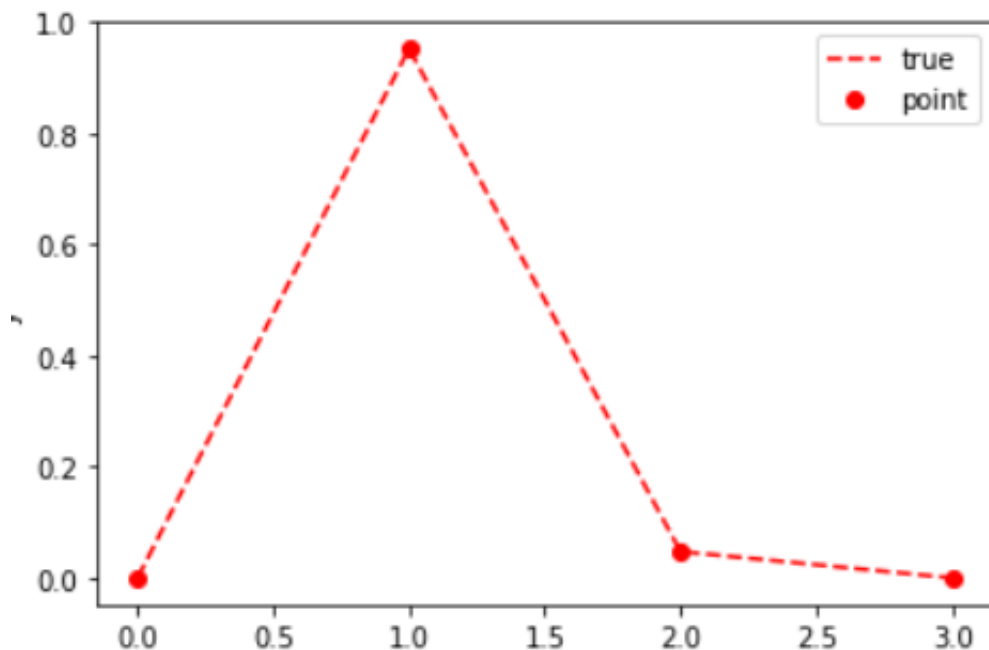
Требуется дополнительно настраивать гиперпараметр (уровень наклона при отрицательных значениях), что требует определенных усилий.

Активационные функции

Функция Softmax

$$P(y = t \mid z_t^{(i)}) = \sigma_{\text{softmax}}(z_t^{(i)}) = \frac{e^{z_t^{(i)}}}{\sum_{j=1}^k e^{z_j^{(i)}}}$$

$$t \in \{j \dots k\}$$



Softmax — это обобщение логистической функции для многомерного случая.

Применяется для задач классификации, когда количество возможных классов больше двух.

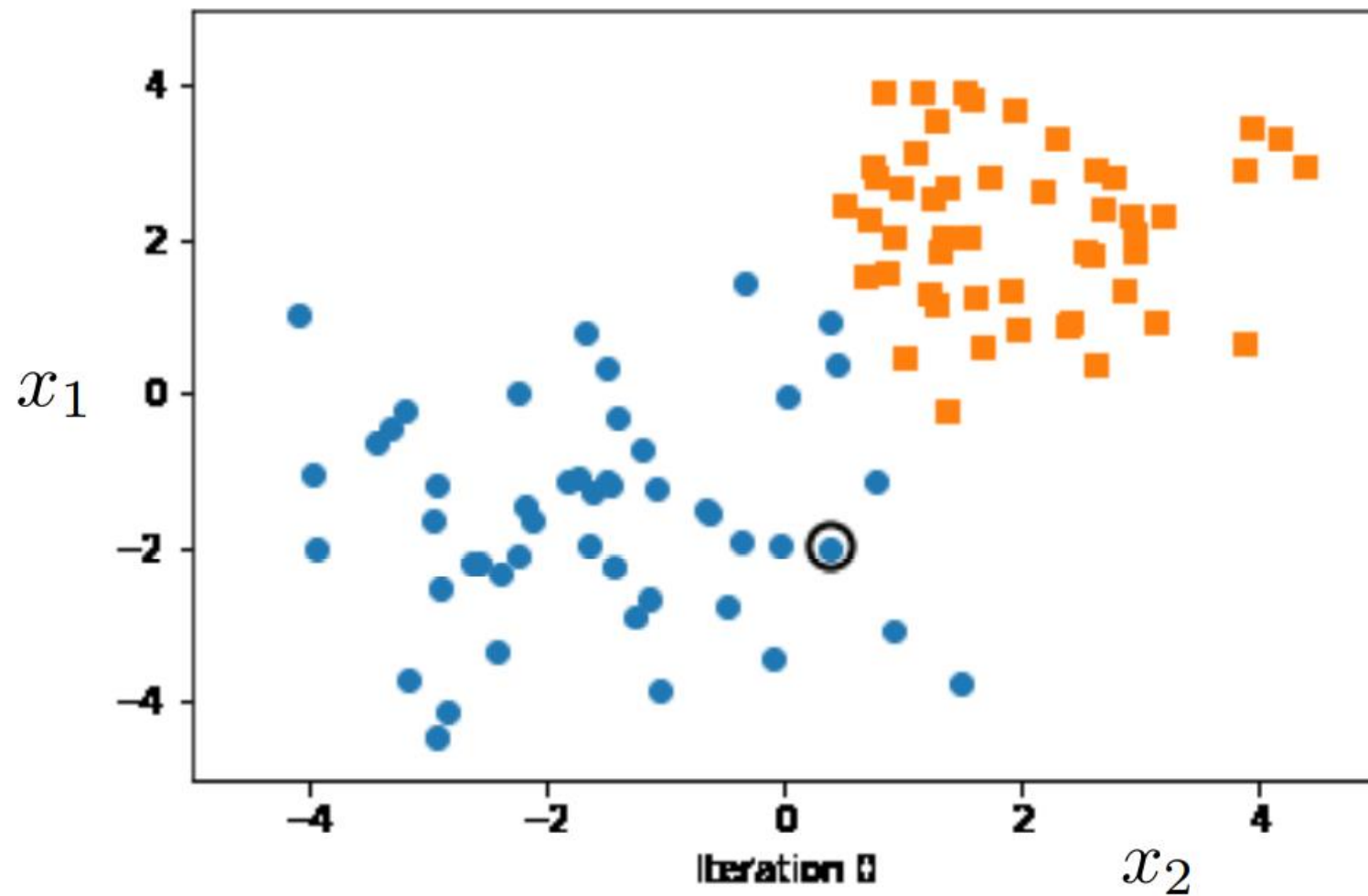
Координаты вектора при этом трактуются как вероятности того, что объект принадлежит к классу i .

Многомерная классификация.

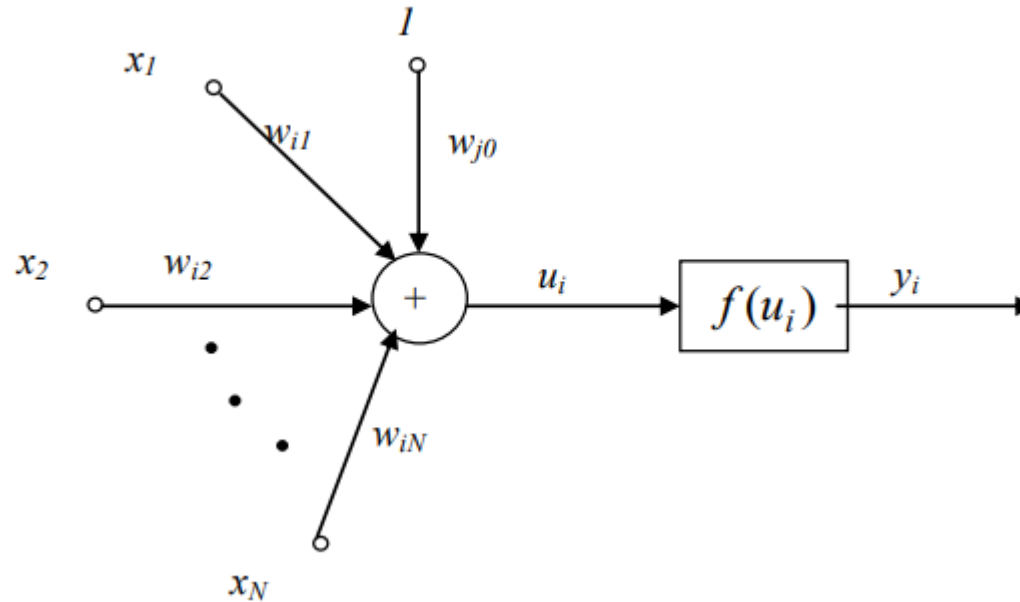
1. История открытия нейронных сетей.
2. Математическая и биологическая модель нейрона.
3. Функция активации, виды функций активации.
- 4. Обучение нейрона**

- **Модели обучения:**
- **Обучение по правилу Хебба**
- **Обучение на основе коррекции ошибок**
- **Обучение с использованием памяти**
- **Конкурентное обучение**
- **Метод Больцмана**

Обучение нейрона

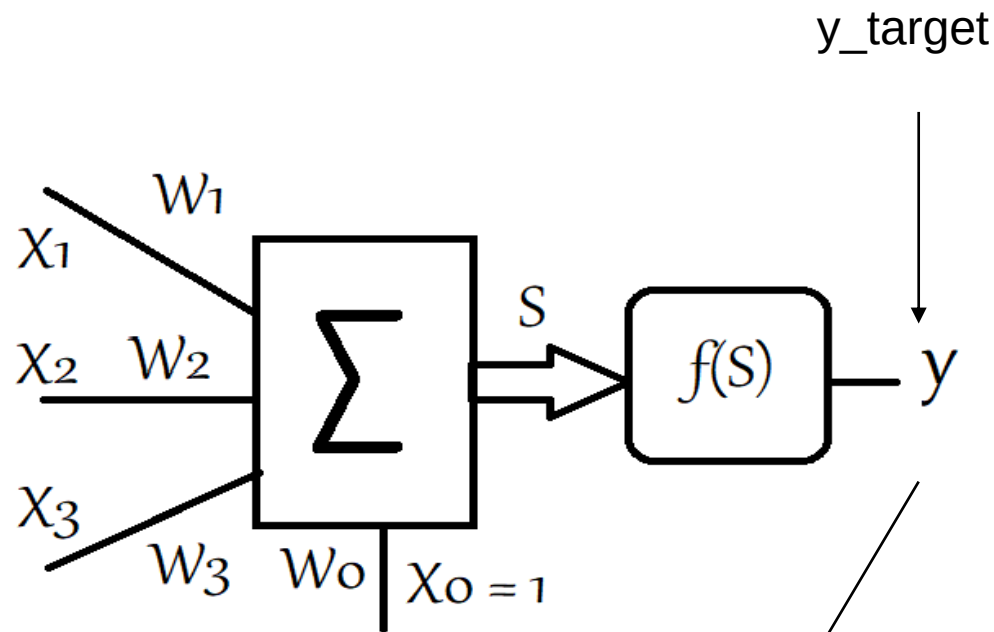


Обучение по правилу Хебба (самоорганизующееся обучение)



1. Если два нейрона по обе стороны синапса (соединения) активизируются одновременно (т.е. синхронно), то прочность этого соединения возрастает.
2. Если два нейрона по обе стороны синапса активизируются асинхронно, то такой синапс ослабляется или вообще отмирает.

Обучение по правилу Хебба



$y < y_{\text{target}}$ $x=1$, w_i — увеличить
 $y > y_{\text{target}}$ $x=1$, w_i — уменьшить

$$x_i \in \{0,1\}$$

$$y = \begin{cases} 1, s \geq 0 \\ -1, s < 0 \end{cases}$$

$$y \in \{-1,1\}$$

$$\Delta w_{ij} = \eta d_i y_j$$

$$w_{ij}(t+1) = w_{ij}(t) + \Delta w_{ij}(t)$$

$$w_{ij}(t+1) = w_{ij}(t) + (1 - \gamma) \Delta w_{ij}(t)$$

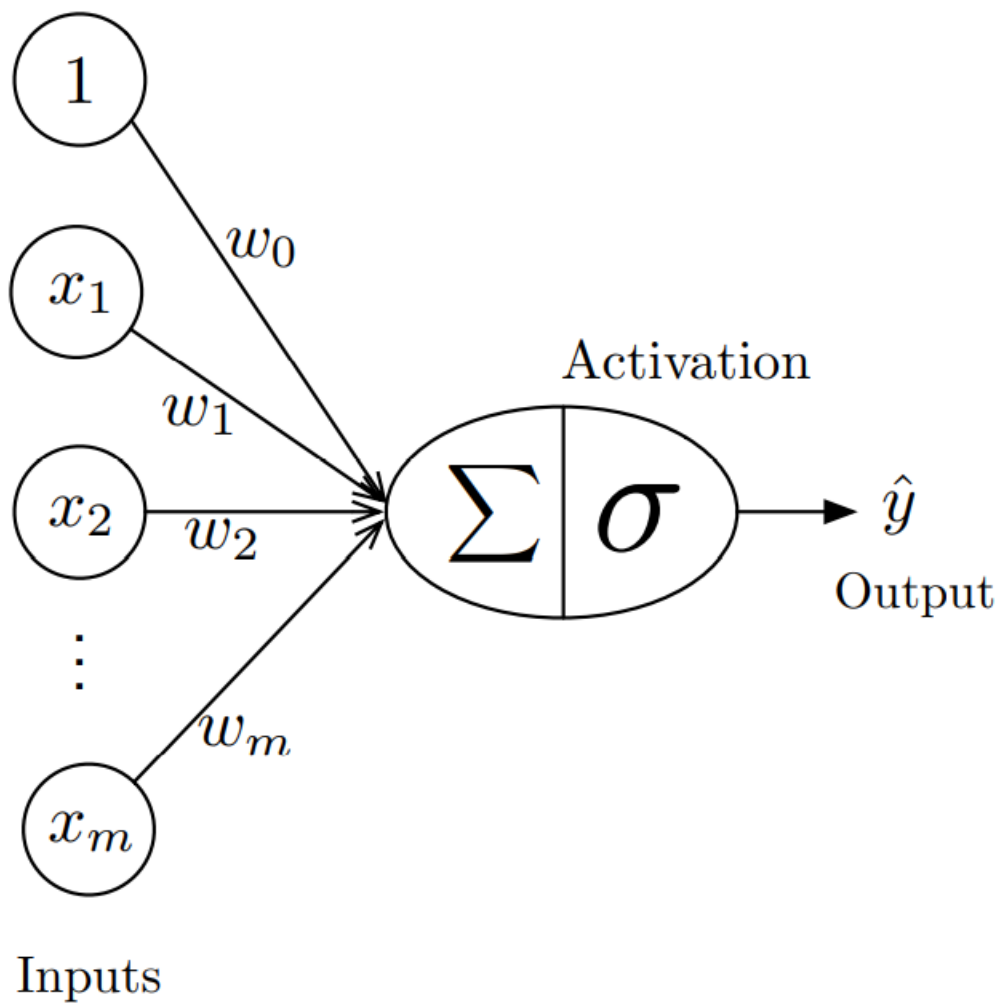
$$\gamma < 0.1 \cdot y$$

$$y_i = \sum_{j=0}^N w_{ij} x_j = w^T x = x^T w$$

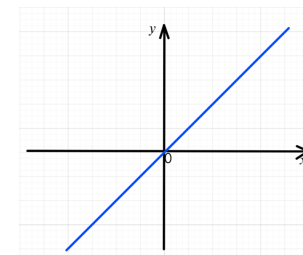
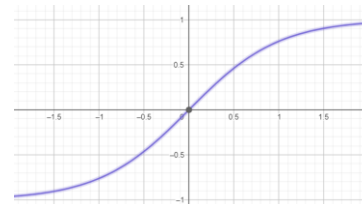
$$\Delta w = \eta xy$$

$$\Delta w = Cw \quad C = xx^T$$

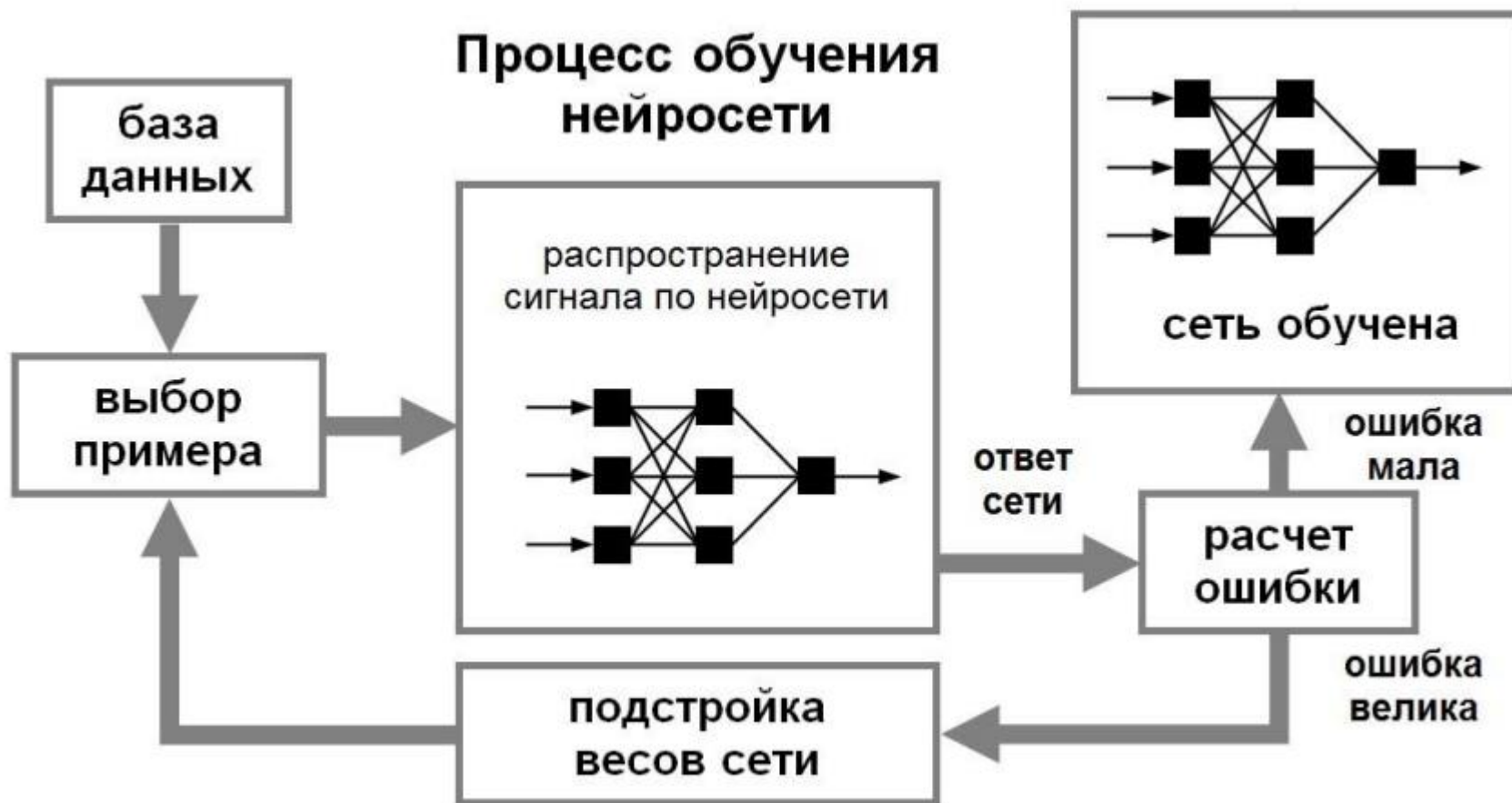
Математическая модель нейрона



$$\sigma \left(\sum_{i=0}^m x_i w_i \right) = \sigma (\mathbf{x}^T \mathbf{w}) = \hat{y}$$



Обучение сети



Обучение на основе коррекции ошибок

Алгоритм основан на применении сигнала ошибки для коррекции синаптических весов определяется функцией стоимости, используемой конкретным методом оптимизации

$$D : \{x_i, y_i, i=1, n\},$$

$$y_i = \sum_{j=1}^m w_{ji} x_{ji},$$

w_{ji} — синаптические веса нейронов в j -й момент времени.

$$Y = X^T W$$

$$e_i = y_i - \tilde{y}_i \text{ — сигнал ошибки}$$

- Методы безусловной оптимизации.
- Линейный фильтр, построенный по методу МНК
- Алгоритм минимизации среднеквадратической ошибки

Коррекция синаптических весов

$$D : \{x_i, y_i, i=1, n\},$$

$$y_i = \sum_{j=1}^m w_{ji} x_{ji},$$

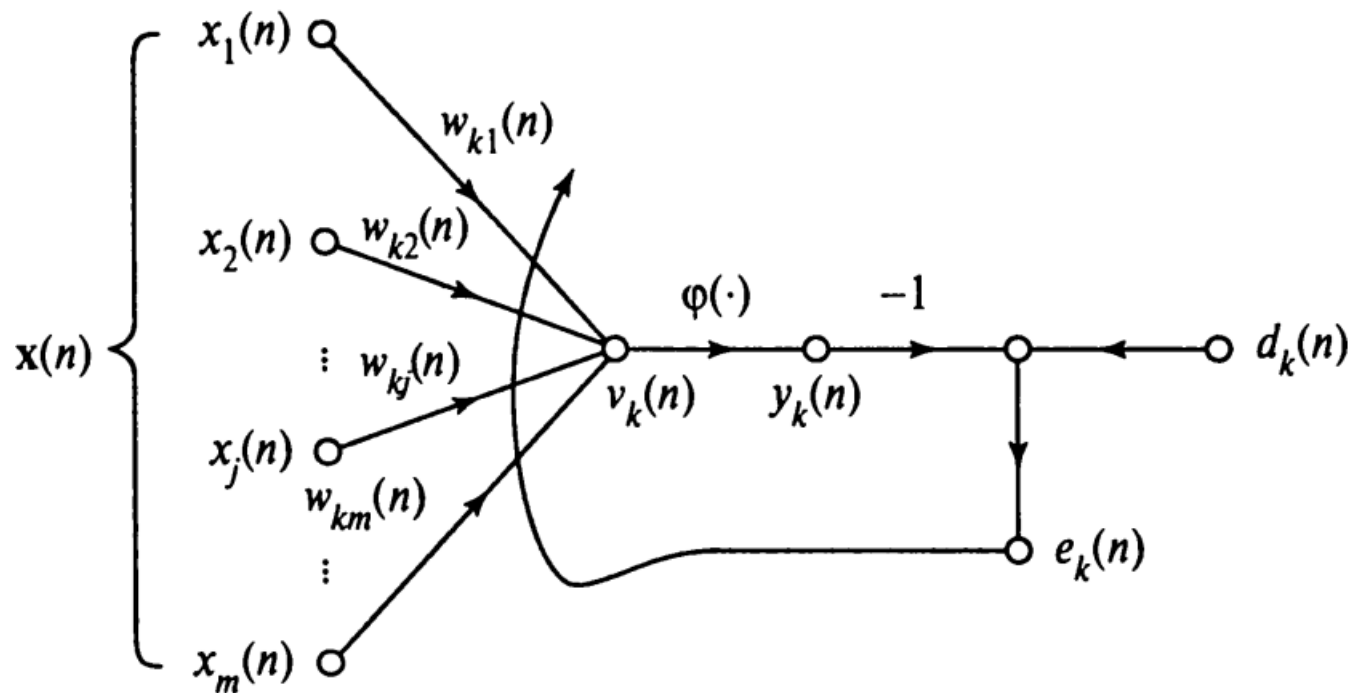
w_{ji} — синаптические веса нейронов в j -й момент времени.

$$Y = X^T W$$

$$e_i = y_i - \tilde{y}_i - \text{сигнал ошибки}$$

$$dw_i = -n (y(X_j) - d_j) x_i$$

$$w_i = w_i + dw_i$$



Конкурентное обучение WTA

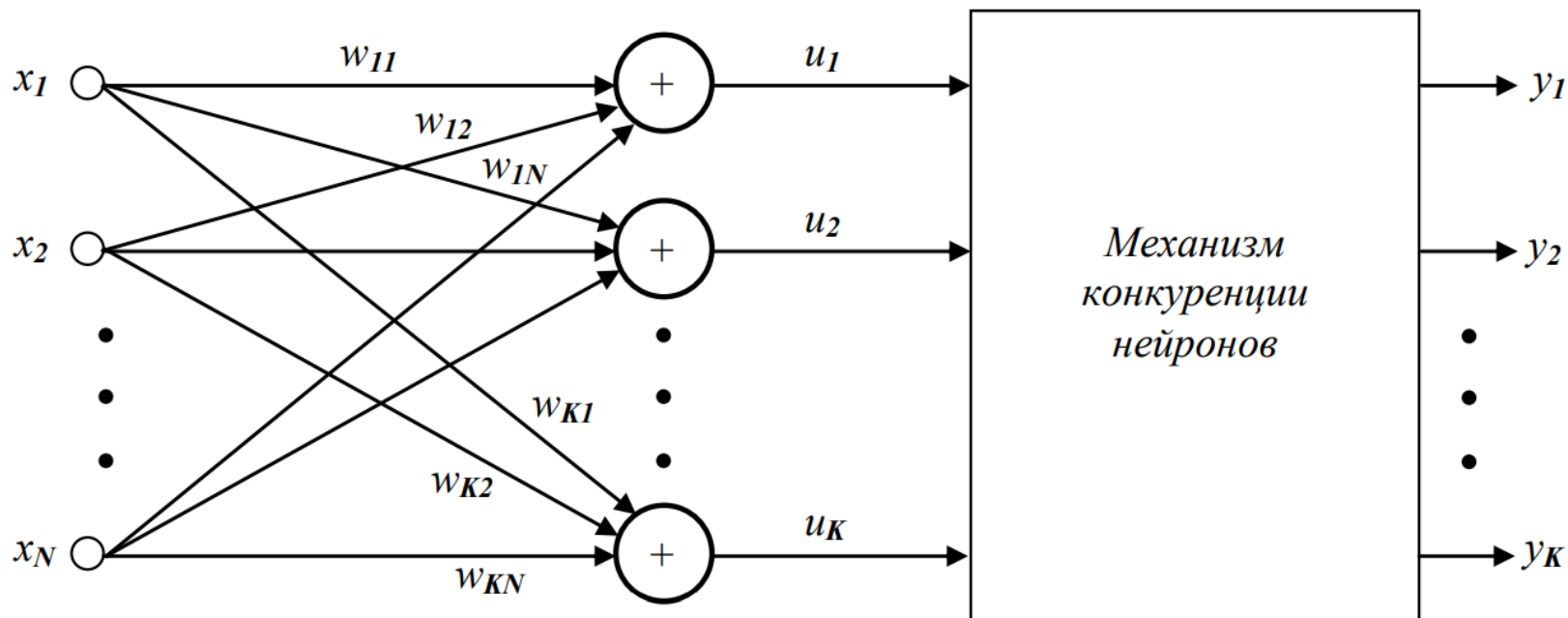
$$D : \{x_i, y_i, i=1, n\},$$

$$y_i = \sum_{j=1}^m w_{ji} x_{ji},$$

w_{ji} — синаптические веса нейронов в j -й момент времени.

$$Y = X^T W$$

$$\Delta w_{ij}(t+1) = w_{ij}(t) + \eta(x_j - w_{ij}(t))$$



Машина Больцмана

Принцип максимального правдоподобия

$$D : \{x_i, y_i, i=1, n\},$$
$$y_i = \sum_{j=1}^m w_{ji} x_{ji},$$

w_{ji} — синаптические веса нейронов в j -й момент времени.

$$Y = X^T W$$

$$E(\mathbf{x}) = -\frac{1}{2} \sum_i \sum_{j, i \neq j} w_{ji} x_i x_j$$

$$L(w, q) = \ln \prod_{i=1, N} p(X_i) = \sum_{i=1, N} \ln \sum_{j=1, K} w_j F(X_i, q_j)$$

Методы оптимизации функции стоимости

Алгоритм применения сигнала ошибки для коррекции синаптических весов определяется функцией стоимости, используемой конкретным методом оптимизации

$$D : \{x_i, y_i, i=1, n\},$$

$$y_i = \sum_{j=1}^m w_{ji} x_{ji},$$

w_{ji} — синаптические веса нейронов в j -й момент времени.

$$Y = X^T W$$

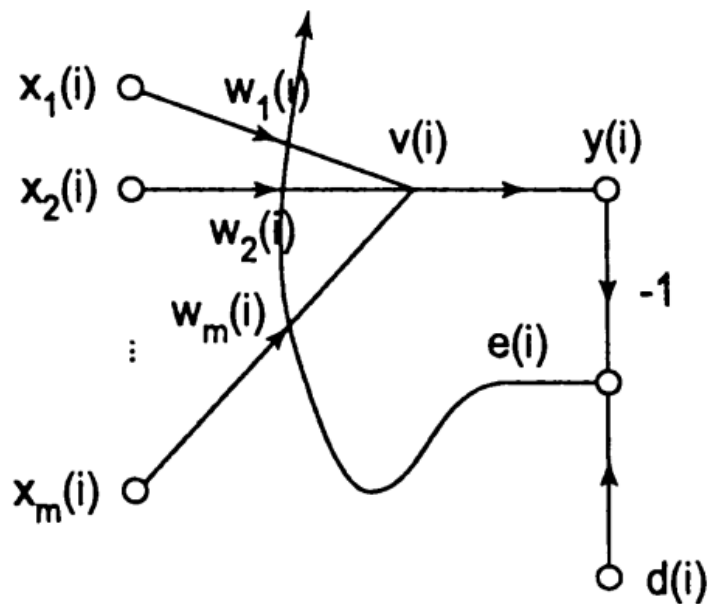
$$e_i = y_i - \tilde{y}_i \text{ — сигнал ошибки}$$

- Методы безусловной оптимизации.
- Линейный фильтр, построенный по методу МНК
- Алгоритм минимизации среднеквадратической ошибки

Методы безусловной оптимизации.

1. Метод наискорейшего спуска
2. Метод Ньютона
3. Метод Ньютона-Гаусса

Задача адаптивной фильтрации



Граф прохождения сигнала

Задача адаптивной фильтрации

Поведение системы описывается набором данных

$$T : \{x_i, y_i, i=1, n\}$$

$y_i = \sum_{j=1}^m w_{ji} x_{ji}$, W_{ji} – синаптические веса нейронов в j -й момент времени.

$$Y = X^T W$$

$e_i = y_i - \tilde{y}_i$ – сигнал ошибки

Функционал ошибок

$$E = \frac{1}{2} (Y - \widetilde{Y})^2$$

Задача – выбор вектора W , минимизирующего ошибку E

Методы безусловной оптимизации.

$E(w)$ – непрерывно дифференцируемая функция стоимости.

Требуется найти w^* такое, что $E(w^*) \leq E(w)$

Необходимо решить задачу безусловной оптимизации: Минимизировать функцию стоимости $E(w)$ по отношению к вектору весов.

$E(w^*) \rightarrow \min$ Необходимое условие оптимальности
 $\nabla E(w^*) = 0$

$$\nabla = \left[\frac{\partial}{\partial w_1}, \dots, \frac{\partial}{\partial w_m} \right]^T$$

$\nabla E(w)$ - вектор градиента функции стоимости.

$$\nabla E(w) = \left[\frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_m} \right]^T$$

Алгоритм последовательного спуска:

Начиная с исходного значения w_0 генерируется последовательность векторов весовых коэффициентов $w_1, w_2, \dots$, таких, что с каждой итерацией алгоритма значение функции стоимости уменьшается $E(w_{n+1}) < E(w_n)$

Решение сойдется к оптимальному w^*

Методы безусловной оптимизации. Метод наискорейшего спуска

$E(\mathbf{w})$ – непрерывно дифференцируемая функция стоимости.

Требуется найти $\mathbf{w}^*$ такое, что

$$E(\mathbf{w}^*) \leq E(\mathbf{w})$$

Начиная с исходного значения $\mathbf{w}_0$ генерируется последовательность векторов весовых коэффициентов $\mathbf{w}_1, \mathbf{w}_2, \dots$, таких, что с каждой итерацией алгоритма значение функции стоимости уменьшается $E(\mathbf{w}_{n+1}) < E(\mathbf{w}_n)$. Решение сойдется к оптимальному $\mathbf{w}^*$.

Пусть $E(\mathbf{w})$ – решение,

Рассмотрим окрестность $E(\mathbf{w} + \mathbf{p})$ и разложим в ряд Тейлора

$\mathbf{g}(\mathbf{w})$ - вектор градиента функции стоимости.

$$\begin{aligned} \mathbf{w}(n+1) &= \mathbf{w}(n) + \Delta \mathbf{w}(n) \quad (1) \\ \Delta \mathbf{w}(n) &= -\eta (\mathbf{y}(\mathbf{X}_j) - \mathbf{d}_j) \mathbf{x}_i = -\eta \mathbf{g}(\mathbf{w}) \end{aligned}$$

$$E(\mathbf{w}(n+1)) = E(\mathbf{w}(n)) + [\mathbf{g}(\mathbf{w})]^T \Delta \mathbf{w}(n),$$

$$E(\mathbf{w}(n+1)) = E(\mathbf{w}(n)) - \eta [\mathbf{g}(\mathbf{w})]^T \mathbf{g}(n), = E(\mathbf{w}(n)) - \eta \|\mathbf{g}(\mathbf{w})\|^2,$$

$$\mathbf{g}(\mathbf{w}) = \left[\frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_m} \right]^T$$

Точкой решения $\mathbf{w}^*$ будем считать точку, в которой достигается минимум целевой функции $E(\mathbf{w})$ и $\mathbf{g}(\mathbf{w}^*) = 0$.

Методы безусловной оптимизации. Метод Ньютона

$E(w)$ – непрерывно дифференцируемая функция стоимости.

Требуется найти w^* такое, что

$$E(w^*) \leq E(w)$$

Начиная с исходного значения w_0 генерируется последовательность векторов весовых коэффициентов $w_1, w_2, \dots$, таких, что с каждой итерацией алгоритма значение функции стоимости уменьшается $E(w_{n+1}) < E(w_n)$. Решение сойдется к оптимальному w^* .

Пусть $E(w)$ – решение,

Рассмотрим окрестность $E(w+p)$ и разложим $g(w)$ – вектор градиента функции стоимости.

$$E(w+p) = E(w) + [g(w)]^T p + \frac{1}{2} p^T H(w) p + \dots,$$

$$w(n+1) = w(n) + \Delta w(n) \quad (1)$$

$$\Delta E w(n) = E(w(n+1)) - E(w(n)) = [g(w)]^T \Delta w(n) + 1/2 [\Delta$$

$$g(n) + H(n) \Delta(w(n)) = 0$$

$$g(w) = \left[\frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_m} \right]^T$$

$$(\Delta w(n)) = -[H(n)]^{-1} g(n)$$

$$w(n+1) = w(n) + \Delta w(n) = w(n) - [H(n)]^{-1} g(n)$$

$$H(w) = \begin{bmatrix} \frac{\partial^2 E}{\partial w_1 \partial w_1} & \dots & \frac{\partial^2 E}{\partial w_1 \partial w_n} \\ \dots & \dots & \dots \\ \frac{\partial^2 E}{\partial w_n \partial w_1} & \dots & \frac{\partial^2 E}{\partial w_n \partial w_n} \end{bmatrix}$$

Точкой решения w^* будем считать точку, в которой достигается минимум целевой функции $E(w)$ и $g(w) = 0$.

Методы безусловной оптимизации. Метод Гаусса-Ньютона

Метод Гаусса-Ньютона применяется для минимизации функции стоимости

$$E(w) = \frac{1}{2} \sum_{i=1}^n e_i^2$$

Сигнал ошибки ищем в виде

$$e'(i, n) = e(i) + \left[\frac{\partial e(i)}{\partial w} \right]_{w=w(n)}^T$$

В матричном виде

$$e'(i, n) = e(n) + J(n)(w - w(n))$$

$$e(n) = [e(1), e(2), \dots, e(n)]$$

$J(n)$ – матрица Якобиан ошибки (транспонированная матрица градиента $\nabla E(n)$)

$$J(n) = \begin{bmatrix} \frac{\partial e(1)}{\partial w_1} & \dots & \frac{\partial e(n)}{\partial w_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial e(1)}{\partial w_n} & \dots & \frac{\partial e(n)}{\partial w_n} \end{bmatrix}$$

$$w(n+1) = \arg \min \left\{ \frac{1}{2} \|e'(n, w)\|^2 \right\}$$

Методы безусловной оптимизации. Метод Гаусса-Ньютона

Для оценки Евклидовой нормы получим

$$\begin{aligned} \frac{1}{2} \|e'(n, w)\|^2 \\ = \frac{1}{2} \|e'(n)\|^2 + e^T(\mathbf{n})\mathbf{J}(n)(w - w(n)) + \frac{1}{2} (w - w(n))^T \mathbf{J}^T(\mathbf{n})\mathbf{J}(n)(w - w(n)) \end{aligned}$$

Продифференцируем по w

$$\mathbf{J}^T(\mathbf{n})\mathbf{e}(n) + \mathbf{J}^T(\mathbf{n})\mathbf{J}(n)(w - w(n)) = 0$$

Формула Гаусса-Ньютона

$$w(n + 1) = w(n) - (\mathbf{J}^T(\mathbf{n})\mathbf{J}(n))^{-1}(\mathbf{J}^T(\mathbf{n})\mathbf{e}(n))$$

Алгоритм минимизации среднеквадратической ошибки (Least-mean square)

$$T : \{x_i, y_i, i=1, n\},$$

$y_i = \sum_{j=1}^m w_{ji} x_{ji}$, w_{ji} – синаптические веса нейронов в j -й момент времени.

$$Y = X^T W$$

$e_i = y_i - \tilde{y}_i$ – сигнал ошибки

Функционал ошибок

$$E = \frac{1}{2} (Y - \widetilde{Y})^2$$

Задача – выбор вектора W , минимизирующего ошибку E

Алгоритм минимизации среднеквадратической ошибки (Least-mean square)

Анализ сходимости алгоритма.

- Векторы входных сигналов x_i , являются статистически независимыми друг от друга.
- В момент времени n вектор x_n , является статистически независимым от всех предыдущих откликов. Y_{n-1} ,
- В момент времени n желаемый отклик y_n зависит от вектора x_n является статистически независимым от всех предыдущих откликов. Y_{n-1} ,
- Вектор входного сигнала и желаемый отклик выбираются из множества, подчиняющегося распределению Гаусса.

Алгоритм сходится, при условии:

$$0 < \eta < \frac{2}{\lambda_{\max}}$$

$\lambda_{\max}$ – наибольшее собственное значение матрицы корреляции входных векторов.

Алгоритм минимизации среднеквадратической ошибки (Least-mean square)

Недостатки:

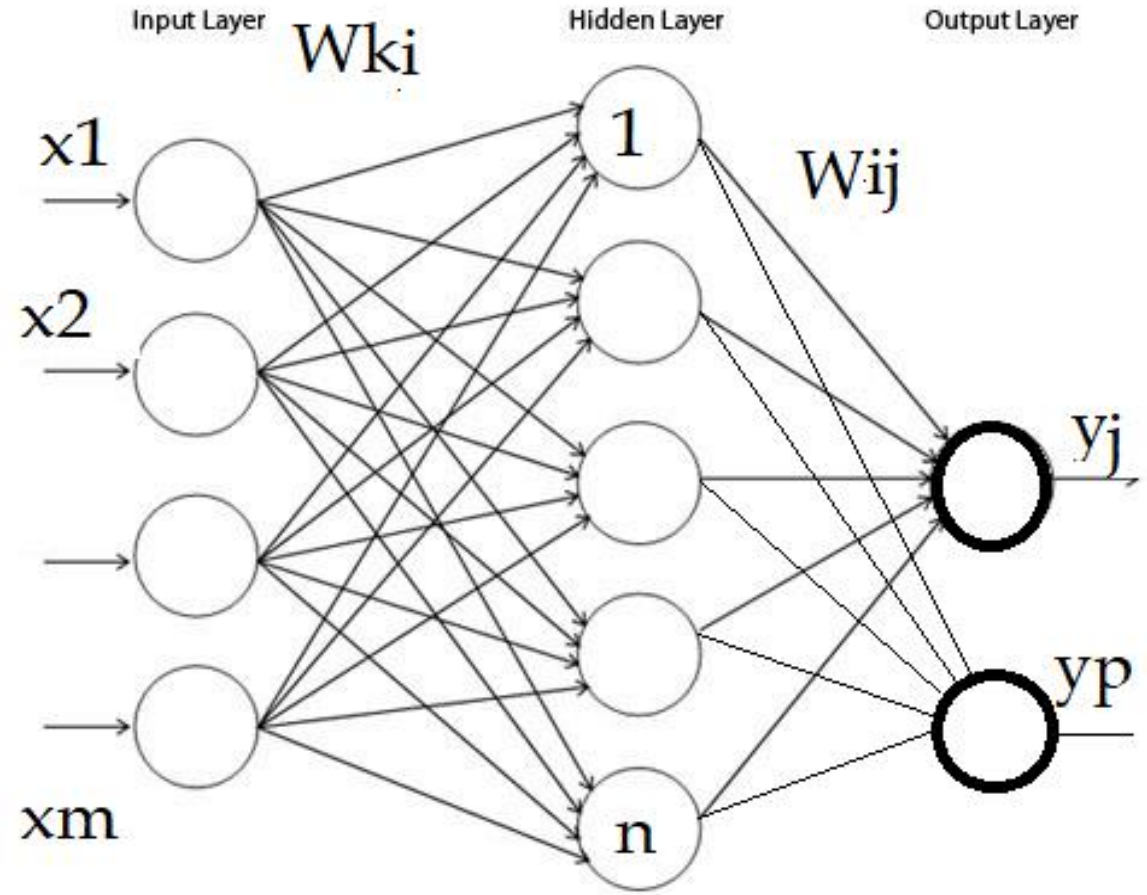
- Низкая скорость сходимости.
- Чувствительность к изменению собственных значений матрицы входных сигналов.

Нейронная сеть

Слой

Множество слоев

$$y_j = f_j \left(\sum_{i=0, n} w_{ij} f_i \left(\sum_{k=0, m} w_{ki} x_k \right) \right)$$



Обобщенное Дельта-правило

Алгоритм минимизации среднеквадратической ошибки
(Least-mean square)

$$x_i \in [0,1]$$

$$y = f(S), S = \sum_{i=0,n} w_i x_i, x_0 = 1$$

$$y \in [0,1]$$

$$E = \frac{1}{2} (y(S) - \hat{y})^2$$

$$\frac{\partial E}{\partial w_i} = (y(S) - \hat{y}) \frac{\partial f(S)}{\partial w_i} \frac{\partial S(X, W)}{\partial w_i} = (y(S) - \hat{y}) x_i \frac{\partial f(S)}{\partial w_i}$$

$$w_i = w_i - \eta (y(S) - \hat{y}) x_i f'(S)$$

Сигмоид Дельта-правило

$$x_i \in [0,1]$$

$$y = \frac{1}{1 + e^{-S/\lambda}}, S = \sum_{i=0,n} w_i x_i, x_0 = 1$$

$$y \in [0,1]$$

$$E = \frac{1}{2} (y(S) - \hat{y})^2$$

$$\frac{\partial E}{\partial w_i} = (y(S) - \hat{y}) x_i \frac{\partial f(S)}{\partial w_i} = (y(S) - \hat{y}) x_i f(S) (1 - f(S))$$

$$w_i = w_i - \eta (y(S) - \hat{y}) x_i f(S) (1 - f(S))$$

$$w_i = w_i - \eta (y(S(X, W)) - \hat{y}) x_i y(S(X, W)) (1 - y(S(X, W)))$$

Дельта-правило

Обучение нейрона по Дельта-правилу

1. Задать начальное состояние
2. Взять пример j из набора примеров
3. Вычислить выход нейрона для этого примера
4. Вычислить коррекцию синаптических весов для всех входов x_i
$$dw_i = -\eta (y(X_j) - d_j) x_i$$
5. Вычислить новый уровень w_i
$$w_i = w_i + dw_i$$
6. Перейти к новому примеру в п.2
7. Если просмотрели все примеры, то проверить условие конца (ошибка модели или число повторов):
 - условие выполнено - конец
 - условие не выполнено - начинаем просматривать примеры и изменять веса сначала ($j=0$) и к п.2.

Эпоха - просмотр всех примеров из набора